

FOLDER: services/api-gateway/

FILE: http.go

```
package main

import (
    "encoding/json"
    "io"
    "log"
    "net/http"
    "ride-sharing/services/api-gateway/grpc_clients"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/env"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/tracing"

    "github.com/stripe/stripe-go/v81"
    "github.com/stripe/stripe-go/v81/webhook"
)

var tracer = tracing.GetTracer("api-gateway")

func handleTripStart(w http.ResponseWriter, r *http.Request) {
    ctx, span := tracer.Start(r.Context(), "handleTripStart")
    defer span.End()

    var reqBody startTripRequest
    if err := json.NewDecoder(r.Body).Decode(&reqBody); err != nil {
        http.Error(w, "failed to parse JSON data", http.StatusBadRequest)
        return
    }

    defer r.Body.Close()

    // Why we need to create a new client for each connection:
    // because if a service is down, we don't want to block the whole application
    // so we create a new client for each connection
    tripService, err := grpc_clients.NewTripServiceClient()
    if err != nil {
        log.Fatal(err)
    }

    // Don't forget to close the client to avoid resource leaks!
    defer tripService.Close()

    trip, err := tripService.Client.CreateTrip(ctx, reqBody.toProto())
    if err != nil {
        log.Printf("Failed to start a trip: %v", err)
        http.Error(w, "Failed to start trip", http.StatusInternalServerError)
        return
    }

    response := contracts.APIResponse{Data: trip}

    writeJSON(w, http.StatusCreated, response)
}

func handleTripPreview(w http.ResponseWriter, r *http.Request) {
    ctx, span := tracer.Start(r.Context(), "handleTripPreview")
    defer span.End()

    var reqBody previewTripRequest
    if err := json.NewDecoder(r.Body).Decode(&reqBody); err != nil {
        http.Error(w, "failed to parse JSON data", http.StatusBadRequest)
        return
    }

    defer r.Body.Close()

    // validation
    if reqBody.UserID == "" {
        http.Error(w, "user ID is required", http.StatusBadRequest)
        return
    }

    // Why we need to create a new client for each connection:
    // because if a service is down, we don't want to block the whole application
    // so we create a new client for each connection
    tripService, err := grpc_clients.NewTripServiceClient()
    if err != nil {
        log.Fatal(err)
    }

    // Don't forget to close the client to avoid resource leaks!
    defer tripService.Close()

    tripPreview, err := tripService.Client.PreviewTrip(ctx, reqBody.toProto())
    if err != nil {
        log.Printf("Failed to preview a trip: %v", err)
        http.Error(w, "Failed to preview trip", http.StatusInternalServerError)
        return
    }

    response := contracts.APIResponse{Data: tripPreview}

    writeJSON(w, http.StatusCreated, response)
}

func handleStripeWebhook(w http.ResponseWriter, r *http.Request, rb *messaging.RabbitMQ) {
    ctx, span := tracer.Start(r.Context(), "handleStripeWebhook")
    defer span.End()

    body, err := io.ReadAll(r.Body)
    if err != nil {
        http.Error(w, "Failed to read request body", http.StatusInternalServerError)
        return
    }
    defer r.Body.Close()

    webhookKey := env.GetString("STRIPE_WEBHOOK_KEY", "")
    if webhookKey == "" {
        log.Printf("Webhook key is required")
        return
    }

    event, err := webhook.ConstructEventWithOptions(
        body,
        r.Header.Get("Stripe-Signature"),
        webhookKey,
        webhook.ConstructEventOptions{
            IgnoreAPIVersionMismatch: true,
        },
    )
    if err != nil {
        log.Printf("Error verifying webhook signature: %v", err)
        http.Error(w, "Invalid signature", http.StatusBadRequest)
        return
    }
}
```

```
log.Printf("Received Stripe event: %v", event)

switch event.Type {
case "checkout.session.completed":
    var session stripe.CheckoutSession

    err := json.Unmarshal(event.Data.Raw, &session)
    if err != nil {
        log.Printf("Error parsing webhook JSON: %v", err)
        http.Error(w, "Invalid payload", http.StatusBadRequest)
        return
    }

    payload := messaging.PaymentStatusUpdatedData{
        TripID: session.Metadata["trip_id"],
        UserID: session.Metadata["user_id"],
        DriverID: session.Metadata["driver_id"],
    }

    payloadBytes, err := json.Marshal(payload)
    if err != nil {
        log.Printf("Error marshalling payload: %v", err)
        http.Error(w, "Failed to marshal payload", http.StatusInternalServerError)
        return
    }

    message := contracts.AmqpMessage{
        OwnerID: session.Metadata["user_id"],
        Data: payloadBytes,
    }

    if err := rb.PublishMessage(
        ctx,
        contracts.PaymentEventSuccess,
        message,
    ); err != nil {
        log.Printf("Error publishing payment event: %v", err)
        http.Error(w, "Failed to publish payment event", http.StatusInternalServerError)
        return
    }
}

Error)
}
```

FILE: json.go

```
package main

import (
    "encoding/json"
    "net/http"
)

func writeJSON(w http.ResponseWriter, status int, data any) error {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    return json.NewEncoder(w).Encode(data)
}
```

FILE: main.go

```
package main

import (
    "context"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "ride-sharing/shared/env"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/tracing"
)

var (
    httpAddr = env.GetString("HTTP_ADDR", ":8081")
    rabbitMQURI = env.GetString("RABBITMQ_URI", "amqp://guest:guest@rabbitmq:5672/")
)

func main() {
    log.Println("Starting API Gateway")

    // Initialize Tracing
    tracerCfg := tracing.Config{
        ServiceName: "api-gateway",
        Environment: env.GetString("ENVIRONMENT", "development"),
        JaegerEndpoint: env.GetString("JAEGER_ENDPOINT", "http://jaeger:14268/api/traces"),
    }

    sh, err := tracing.InitTracer(tracerCfg)
    if err != nil {
        log.Fatalf("Failed to initialize the tracer: %v", err)
    }

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    defer sh(ctx)

    mux := http.NewServeMux()

    // RabbitMQ connection
    rabbitmq, err := messaging.NewRabbitMQ(rabbitMQURI)
    if err != nil {
        log.Fatal(err)
    }
    defer rabbitmq.Close()

    log.Println("Starting RabbitMQ connection")

    mux.Handle("POST /trip/preview", tracing.WrapHandlerFunc(enableCORS(handleTripPreview), "/trip/preview"))
    mux.Handle("POST /trip/start", tracing.WrapHandlerFunc(enableCORS(handleTripStart), "/trip/start"))
    mux.Handle("/ws/drivers", tracing.WrapHandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        handleDriversWebSocket(w, r, rabbitmq)
    }, "/ws/drivers"))
    mux.Handle("/ws/riders", tracing.WrapHandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        handleRidersWebSocket(w, r, rabbitmq)
    }, "/ws/riders"))
    mux.Handle("/webhook/stripe", tracing.WrapHandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        handleStripeWebhook(w, r, rabbitmq)
    }, "/webhook/stripe"))

    server := &http.Server{
```

```

    Addr:    httpAddr,
    Handler: mux,
}

serverErrors := make(chan error, 1)

go func() {
    log.Printf("Server listening on %s", httpAddr)
    serverErrors <- server.ListenAndServe()
}()

shutdown := make(chan os.Signal, 1)
signal.Notify(shutdown, os.Interrupt, syscall.SIGTERM)

select {
case err := <-serverErrors:
    log.Printf("Error starting the server: %v", err)

case sig := <-shutdown:
    log.Printf("Server is shutting down due to %v signal", sig)

    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        log.Printf("Could not stop the server gracefully: %v", err)
        server.Close()
    }
}

}

```

FILE: middleware.go

```

package main

import "net/http"

func enableCORS(handler http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "*")
        w.Header().Set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTION")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type, Authorization")

        // allow preflight requests from the browser API
        if r.Method == "OPTIONS" {
            w.WriteHeader(http.StatusOK)
            return
        }

        handler(w, r)
    }
}

```

FILE: types.go

```

package main

import (
    pb "ride-sharing/shared/proto/trip"
    "ride-sharing/shared/types"
)

type previewTripRequest struct {
    UserID      string `json:"userID"`
    Pickup      types.Coordinate `json:"pickup"`
    Destination types.Coordinate `json:"destination"`
}

func (p *previewTripRequest) toProto() *pb.PreviewTripRequest {
    return &pb.PreviewTripRequest{
        UserID: p.UserID,
        StartLocation: &pb.Coordinate{
            Latitude: p.Pickup.Latitude,
            Longitude: p.Pickup.Longitude,
        },
        EndLocation: &pb.Coordinate{
            Latitude: p.Destination.Latitude,
            Longitude: p.Destination.Longitude,
        },
    }
}

type startTripRequest struct {
    RideFareID string `json:"rideFareID"`
    UserID      string `json:"userID"`
}

func (c *startTripRequest) toProto() *pb.CreateTripRequest {
    return &pb.CreateTripRequest{
        RideFareID: c.RideFareID,
        UserID:     c.UserID,
    }
}

```

FILE: ws.go

```

package main

import (
    "encoding/json"
    "log"
    "net/http"
    "ride-sharing/services/api-gateway/grpc_clients"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/proto/driver"
)

var (
    connManager = messaging.NewConnectionManager()
)

func handleRidersWebSocket(w http.ResponseWriter, r *http.Request, rb *messaging.RabbitMQ) {
    conn, err := connManager.Upgrade(w, r)
    if err != nil {
        log.Printf("WebSocket upgrade failed: %v", err)
        return
    }

    defer conn.Close()

    userID := r.URL.Query().Get("userID")
    if userID == "" {
        log.Println("No user ID provided")
        return
    }

    // Add connection to manager
    connManager.Add(userID, conn)
    defer connManager.Remove(userID)
}

```

```

// Initialize queue consumers
queues := []string{
    messaging.NotifyDriverNoDriversFoundQueue,
    messaging.NotifyDriverAssignQueue,
    messaging.NotifyPaymentSessionCreatedQueue,
}

for _, q := range queues {
    consumer := messaging.NewQueueConsumer(rb, connManager, q)

    if err := consumer.Start(); err != nil {
        log.Printf("Failed to start consumer for queue: %s: err: %v", q, err)
    }
}

for {
    _, message, err := conn.ReadMessage()
    if err != nil {
        log.Printf("Error reading message: %v", err)
        break
    }

    log.Printf("Received message: %s", message)
}

}

func handleDriversWebSocket(w http.ResponseWriter, r *http.Request, rb *messaging.RabbitMQ) {
    conn, err := connManager.Upgrade(w, r)
    if err != nil {
        log.Printf("WebSocket upgrade failed: %v", err)
        return
    }

    defer conn.Close()

    userID := r.URL.Query().Get("userID")
    if userID == "" {
        log.Println("No user ID provided")
        return
    }

    packageSlug := r.URL.Query().Get("packageSlug")
    if packageSlug == "" {
        log.Println("No package slug provided")
        return
    }

    // Add connection to manager
    connManager.Add(userID, conn)

    ctx := r.Context()

    driverService, err := grpc_clients.NewDriverServiceClient()
    if err != nil {
        log.Fatal(err)
    }

    // Closing connections
    defer func() {
        connManager.Remove(userID)

        driverService.Client.UnregisterDriver(ctx, &driver.RegisterDriverRequest{
            DriverID: userID,
            PackageSlug: packageSlug,
        })

        driverService.Close()

        log.Println("Driver unregistered: ", userID)
    }()

    driverData, err := driverService.Client.RegisterDriver(ctx, &driver.RegisterDriverRequest{
        DriverID: userID,
        PackageSlug: packageSlug,
    })
    if err != nil {
        log.Printf("Error registering driver: %v", err)
        return
    }

    if err := connManager.SendMessage(userID, contracts.WSMessage{
        Type: contracts.DriverCmdRegister,
        Data: driverData.Driver,
    }); err != nil {
        log.Printf("Error sending message: %v", err)
        return
    }

    // Initialize queue consumers
    queues := []string{
        messaging.DriverCmdTripRequestQueue,
    }

    for _, q := range queues {
        consumer := messaging.NewQueueConsumer(rb, connManager, q)

        if err := consumer.Start(); err != nil {
            log.Printf("Failed to start consumer for queue: %s: err: %v", q, err)
        }
    }

    for {
        _, message, err := conn.ReadMessage()
        if err != nil {
            log.Printf("Error reading message: %v", err)
            break
        }

        type driverMessage struct {
            Type string `json:"type"`
            Data json.RawMessage `json:"data"`
        }

        var driverMsg driverMessage
        if err := json.Unmarshal(message, &driverMsg); err != nil {
            log.Printf("Error unmarshaling driver message: %v", err)
            continue
        }

        // Handle the different message type
        switch driverMsg.Type {
        case contracts.DriverCmdLocation:
            // Handle driver location update in the future
            continue
        case contracts.DriverCmdTripAccept, contracts.DriverCmdTripDecline:
            // Forward the message to RabbitMQ
            if err := rb.PublishMessage(ctx, driverMsg.Type, contracts.AmqpMessage{

```

```
        OwnerID: userID,
        Data:   driverMsg.Data,
    }); err != nil {
        log.Printf("Error publishing message to RabbitMQ: %v", err)
    }
    default:
        log.Printf("Unknown message type: %s", driverMsg.Type)
    }
}
}
```

FOLDER: services/api-gateway/grpc_clients/

FILE: driver_client.go

```
package grpc_clients

import (
    "os"
    pb "ride-sharing/shared/proto/driver"
    "ride-sharing/shared/tracing"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

type driverServiceClient struct {
    Client pb.DriverServiceClient
    conn  *grpc.ClientConn
}

func NewDriverServiceClient() (*driverServiceClient, error) {
    driverServiceURL := os.Getenv("DRIVER_SERVICE_URL")
    if driverServiceURL == "" {
        driverServiceURL = "driver-service:9092"
    }

    dialOptions := append(
        tracing.DialOptionsWithTracing(),
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )

    conn, err := grpc.NewClient(driverServiceURL, dialOptions...)
    if err != nil {
        return nil, err
    }

    client := pb.NewDriverServiceClient(conn)

    return &driverServiceClient{
        Client: client,
        conn:   conn,
    }, nil
}

func (c *driverServiceClient) Close() {
    if c.conn != nil {
        if err := c.conn.Close(); err != nil {
            return
        }
    }
}
```

FILE: trip_client.go

```
package grpc_clients

import (
    "os"
    pb "ride-sharing/shared/proto/trip"
    "ride-sharing/shared/tracing"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

type tripServiceClient struct {
    Client pb.TripServiceClient
    conn  *grpc.ClientConn
}

func NewTripServiceClient() (*tripServiceClient, error) {
    tripServiceURL := os.Getenv("TRIP_SERVICE_URL")
    if tripServiceURL == "" {
        tripServiceURL = "trip-service:9093"
    }

    dialOptions := append(
        tracing.DialOptionsWithTracing(),
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )

    conn, err := grpc.NewClient(tripServiceURL, dialOptions...)
    if err != nil {
        return nil, err
    }

    client := pb.NewTripServiceClient(conn)

    return &tripServiceClient{
        Client: client,
        conn:   conn,
    }, nil
}

func (c *tripServiceClient) Close() {
    if c.conn != nil {
        if err := c.conn.Close(); err != nil {
            return
        }
    }
}
```

FOLDER: services/driver-service/

FILE: gprc_handler.go

```
package main

import (
    "context"
    pb "ride-sharing/shared/proto/driver"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

type driverGrpcHandler struct {
    pb.UnimplementedDriverServiceServer

    service *Service
}
```

```
func NewGrpcHandler(s *grpc.Server, service *Service) {
    handler := &driverGrpcHandler{
        service: service,
    }

    pb.RegisterDriverServiceServer(s, handler)
}

func (h *driverGrpcHandler) RegisterDriver(ctx context.Context, req *pb.RegisterDriverRequest) (*pb.RegisterDriverResponse, error) {
    driver, err := h.service.RegisterDriver(req.GetDriverID(), req.GetPackageSlug())
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to register driver")
    }

    return &pb.RegisterDriverResponse{
        Driver: driver,
    }, nil
}

func (h *driverGrpcHandler) UnregisterDriver(ctx context.Context, req *pb.RegisterDriverRequest) (*pb.RegisterDriverResponse, error) {
    h.service.UnregisterDriver(req.GetDriverID())

    return &pb.RegisterDriverResponse{
        Driver: &pb.Driver{
            Id: req.GetDriverID(),
        },
    }, nil
}
```

FILE: main.go

```
package main

import (
    "context"
    "log"
    "net"
    "os"
    "os/signal"
    "ride-sharing/shared/env"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/tracing"
    "syscall"

    "google.golang.org/grpc"

    "github.com/jaegertracing/jaeger-client-go"
)

var GrpcAddr = ":9092"

func main() {
    // Initialize Tracing
    tracerCfg := tracing.Config{
        ServiceName: "driver-service",
        Environment: env.GetString("ENVIRONMENT", "development"),
        JaegerEndpoint: env.GetString("JAEGER_ENDPOINT", "http://jaeger:14268/api/traces"),
    }

    sh, err := tracing.InitTracer(tracerCfg)
    if err != nil {
        log.Fatalf("Failed to initialize the tracer: %v", err)
    }

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    defer sh(ctx)

    rabbitMqURI := env.GetString("RABBITMQ_URI", "amqp://guest:guest@rabbitmq:5672/")

    go func() {
        sigCh := make(chan os.Signal, 1)
        signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
        <-sigCh
        cancel()
    }()

    lis, err := net.Listen("tcp", GrpcAddr)
    if err != nil {
        log.Fatalf("failed to listen: %v", err)
    }

    svc := NewService()

    // RabbitMQ connection
    rabbitmq, err := messaging.NewRabbitMQ(rabbitMqURI)
    if err != nil {
        log.Fatal(err)
    }
    defer rabbitmq.Close()

    log.Println("Starting RabbitMQ connection")

    // Starting the gRPC server
    grpcServer := grpcServer(tracing.WithTracingInterceptors())
    NewGrpcHandler(grpcServer, svc)

    consumer := NewTripConsumer(rabbitmq, svc)
    go func() {
        if err := consumer.Listen(); err != nil {
            log.Fatalf("Failed to listen to the message: %v", err)
        }
    }()

    log.Printf("Starting gRPC server Driver service on port %s", lis.Addr().String())

    go func() {
        if err := grpcServer.Serve(lis); err != nil {
            log.Printf("failed to serve: %v", err)
            cancel()
        }
    }()

    // wait for the shutdown signal
    <-ctx.Done()
    log.Println("Shutting down the server...")
    grpcServer.GracefulStop()
}
```

FILE: service.go

```
package main

import (
    math "math/rand/v2"
    pb "ride-sharing/shared/proto/driver"
    "ride-sharing/shared/util"
    "sync"

    "github.com/mmcloughlin/geohash"
}
```

```
)

type driverInMap struct {
    Driver *pb.Driver
    // Index int
    // TODO: route
}

type Service struct {
    drivers []*driverInMap
    mu      sync.RWMutex
}

func NewService() *Service {
    return &Service{
        drivers: make([]*driverInMap, 0),
    }
}

func (s *Service) FindAvailableDrivers(packageType string) []string {
    var matchingDrivers []string

    for _, driver := range s.drivers {
        if driver.Driver.PackageSlug == packageType {
            matchingDrivers = append(matchingDrivers, driver.Driver.Id)
        }
    }

    if len(matchingDrivers) == 0 {
        return []string{}
    }

    return matchingDrivers
}

func (s *Service) RegisterDriver(driverId string, packageSlug string) (*pb.Driver, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

    randomIndex := math.IntN(len(PredefinedRoutes))
    randomRoute := PredefinedRoutes[randomIndex]

    randomPlate := GenerateRandomPlate()
    randomAvatar := util.GetRandomAvatar(randomIndex)

    // we can ignore this property for now, but it must be sent to the frontend.
    geohash := geohash.Encode(randomRoute[0][0], randomRoute[0][1])

    driver := &pb.Driver{
        Id:        driverId,
        Geohash:    geohash,
        Location:   &pb.Location{Latitude: randomRoute[0][0], Longitude: randomRoute[0][1]},
        Name:       "Lando Norris",
        PackageSlug: packageSlug,
        ProfilePicture: randomAvatar,
        CarPlate:    randomPlate,
    }

    s.drivers = append(s.drivers, &driverInMap{
        Driver: driver,
    })

    return driver, nil
}

func (s *Service) UnregisterDriver(driverId string) {
    s.mu.Lock()
    defer s.mu.Unlock()

    for i, driver := range s.drivers {
        if driver.Driver.Id == driverId {
            s.drivers = append(s.drivers[:i], s.drivers[i+1:]...)
        }
    }
}

FILE: trip_consumer.go
package main

import (
    "context"
    "encoding/json"
    "log"
    "math/rand"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"

    "github.com/rabbitmq/amqp091-go"
)

type tripConsumer struct {
    rabbitmq *messaging.RabbitMQ
    service  *Service
}

func NewTripConsumer(rabbitmq *messaging.RabbitMQ, service *Service) *tripConsumer {
    return &tripConsumer{
        rabbitmq: rabbitmq,
        service:  service,
    }
}

func (c *tripConsumer) Listen() error {
    return c.rabbitmq.ConsumeMessages(messaging.FindAvailableDriversQueue, func(ctx context.Context, msg amqp091.Delivery) error {
        var tripEvent contracts.AmqpMessage
        if err := json.Unmarshal(msg.Body, &tripEvent); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        var payload messaging.TripEventData
        if err := json.Unmarshal(tripEvent.Data, &payload); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        log.Printf("driver received message: %v", payload)

        switch msg.RoutingKey {
        case contracts.TripEventCreated, contracts.TripEventDriverNotInterested:
            return c.handleFindAndNotifyDrivers(ctx, payload)
        }

        log.Printf("unknown trip event: %v", payload)

        return nil
    })
}
```

```
})

func (c *tripConsumer) handleFindAndNotifyDrivers(ctx context.Context, payload messaging.TripEventData) error {
    suitableIDs := c.service.FindAvailableDrivers(payload.Trip.SelectedFare.PackageSlug)

    log.Printf("Found suitable drivers %v", len(suitableIDs))

    if len(suitableIDs) == 0 {
        // Notify the driver that no drivers are available
        if err := c.rabbitmq.PublishMessage(ctx, contracts.TripEventNoDriversFound, contracts.AmqpMessage{
            OwnerID: payload.Trip.UserID,
        }); err != nil {
            log.Printf("Failed to publish message to exchange: %v", err)
            return err
        }

        return nil
    }

    // Get a random index from the matching drivers
    randomIndex := rand.Intn(len(suitableIDs))

    suitableDriverID := suitableIDs[randomIndex]

    marshalledEvent, err := json.Marshal(payload)
    if err != nil {
        return err
    }

    // Notify the driver about a potential trip
    if err := c.rabbitmq.PublishMessage(ctx, contracts.DriverCmdTripRequest, contracts.AmqpMessage{
        OwnerID: suitableDriverID,
        Data:     marshalledEvent,
    }); err != nil {
        log.Printf("Failed to publish message to exchange: %v", err)
        return err
    }

    return nil
}

FILE: utils.go
package main

import "math/rand"

// Predefined routes for drivers (used for the gRPC Streaming module)
// (these are San Francisco routes, get these coordinates from Google Maps for example and build a custom route if you want)
var PredefinedRoutes = [][][]float64{
    {
        {37.768727753110106, -122.413455977077878},
        {37.77019784198334, -122.41298146942745},
        {37.77163599059948, -122.41125013468515},
        {37.773790702602305, -122.41168660785345},
    },
    {
        {37.78938865879484, -122.42206098118852},
        {37.79112418447625, -122.42238063604239},
        {37.79160600785717, -122.42249902672565},
        {37.79111015076977, -122.42238063603901},
        {37.791797800743794, -122.42253454393163},
        {37.79353326986209, -122.4228837964593},
        {37.7939729779581, -122.42294891128813},
        {37.793739091013016, -122.42474844980799},
        {37.79289241408059, -122.42455310517508},
        {37.792172029382556, -122.4244643121601},
        {37.79264916808487, -122.42616913804746},
    },
    {
        {37.78647766728455, -122.42321282905907},
        {37.78374742447772, -122.42269398620836},
        {37.78300293033823, -122.4225475612199},
        {37.78291035023184, -122.42089295885036},
        {37.78310564009152, -122.41910523857551},
        {37.783340947012974, -122.41759218036147},
        {37.78242075518227, -122.41739703541508},
        {37.78149494160786, -122.41715787460058},
    },
    {
        {37.78149494160786, -122.41715787460058},
        {37.78242075518227, -122.41739703541508},
        {37.783340947012974, -122.41759218036147},
        {37.78310564009152, -122.41910523857551},
        {37.78291035023184, -122.42089295885036},
        {37.78374742447772, -122.42269398620836},
        {37.78647766728455, -122.42321282905907},
        {37.78300293033823, -122.4225475612199},
    },
}

func GenerateRandomPlate() string {
    letters := "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    plate := ""
    for i := 0; i < 3; i++ {
        plate += string(letters[rand.Intn(len(letters))])
    }

    return plate
}
```

FOLDER: services/payment-service/cmd/

FILE: main.go

```
package main

import (
    "context"
    "log"
    "os"
    "os/signal"
    "syscall"

    "ride-sharing/services/payment-service/internal/events"
    "ride-sharing/services/payment-service/internal/infrastructure/stripe"
    "ride-sharing/services/payment-service/internal/service"
    "ride-sharing/services/payment-service/pkg/types"
    "ride-sharing/shared/env"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/tracing"
)

var GrpcAddr = env.GetString("GRPC_ADDR", ":9004")

func main() {
```

```
// Initialize Tracing
tracerCfg := tracing.Config{
    ServiceName: "payment-service",
    Environment: env.GetString("ENVIRONMENT", "development"),
    JaegerEndpoint: env.GetString("JAEGER_ENDPOINT", "http://jaeger:14268/api/traces"),
},

sh, err := tracing.InitTracer(tracerCfg)
if err != nil {
    log.Fatalf("Failed to initialize the tracer: %v", err)
}

ctx, cancel := context.WithCancel(context.Background())
defer cancel()
defer sh(ctx)

rabbitmqURI := env.GetString("RABBITMQ_URI", "amqp://guest:guest@rabbitmq:5672/")

// Setup graceful shutdown
go func() {
    sigCh := make(chan os.Signal, 1)
    signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
    <-sigCh
    cancel()
}()

appURL := env.GetString("APP_URL", "http://localhost:3000")

// Stripe config
stripeCfg := &types.PaymentConfig{
    StripeSecretKey: env.GetString("STRIPE_SECRET_KEY", ""),
    SuccessURL:     env.GetString("STRIPE_SUCCESS_URL", appURL+"?payment=success"),
    CancelURL:      env.GetString("STRIPE_CANCEL_URL", appURL+"?payment=cancel"),
},

if stripeCfg.StripeSecretKey == "" {
    log.Fatalf("STRIPE_SECRET_KEY is not set")
    return
}

// Stripe processor
paymentProcessor := stripe.NewStripeClient(stripeCfg)

// Service
svc := service.NewPaymentService(paymentProcessor)

// RabbitMQ connection
rabbitmq, err := messaging.NewRabbitMQ(rabbitmqURI)
if err != nil {
    log.Fatal(err)
}
defer rabbitmq.Close()

log.Println("Starting RabbitMQ connection")

// Trip Consumer
tripConsumer := events.NewTripConsumer(rabbitmq, svc)
go tripConsumer.Listen()

// Wait for shutdown signal
<-ctx.Done()
log.Println("Shutting down payment service...")
}
```

FOLDER: services/payment-service/internal/domain/**FILE: domain.go**

```
package domain

import (
    "context"

    "ride-sharing/services/payment-service/pkg/types"
)

type Service interface {
    CreatePaymentSession(ctx context.Context, tripID, userID, driverID string, amount int64, currency string) (*types.PaymentIntent, error)
}

type PaymentProcessor interface {
    CreatePaymentSession(ctx context.Context, amount int64, currency string, metadata map[string]string) (string, error)
}
```

FOLDER: services/payment-service/internal/events/**FILE: trip_consumer.go**

```
package events

import (
    "context"
    "encoding/json"
    "log"

    "ride-sharing/services/payment-service/internal/domain"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"

    "github.com/rabbitmq/amqp091-go"
)

type TripConsumer struct {
    rabbitmq *messaging.RabbitMQ
    service domain.Service
}

func NewTripConsumer(rabbitmq *messaging.RabbitMQ, service domain.Service) *TripConsumer {
    return &TripConsumer{
        rabbitmq: rabbitmq,
        service:  service,
    }
}

func (c *TripConsumer) Listen() error {
    return c.rabbitmq.ConsumeMessages(messaging.PaymentTripResponseQueue, func(ctx context.Context, msg amqp091.Delivery) error {
        var message contracts.AmqpMessage
        if err := json.Unmarshal(msg.Body, &message); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        var payload messaging.PaymentTripResponseData
        if err := json.Unmarshal(message.Data, &payload); err != nil {
            log.Printf("Failed to unmarshal payload: %v", err)
            return err
        }
    })
}
```

```
}

switch msg.RoutingKey {
case contracts.PaymentCmdCreateSession:
    if err := c.handleTripAccepted(ctx, payload); err != nil {
        log.Printf("Failed to handle trip accepted: %v", err)
        return err
    }
}

return nil
})

func (c *TripConsumer) handleTripAccepted(ctx context.Context, payload messaging.PaymentTripResponseData) error {
    log.Printf("Handling trip accepted by driver: %s", payload.TripID)

    paymentSession, err := c.service.CreatePaymentSession(
        ctx,
        payload.TripID,
        payload.UserID,
        payload.DriverID,
        int64(payload.Amount),
        payload.Currency,
    )
    if err != nil {
        log.Printf("Failed to create payment session: %v", err)
        return err
    }

    log.Printf("Payment session created: %s", paymentSession.StripeSessionID)

    // Publish payment session created event
    paymentPayload := messaging.PaymentEventSessionCreatedData{
        TripID:    payload.TripID,
        SessionID: paymentSession.StripeSessionID,
        Amount:    float64(paymentSession.Amount) / 100.0, // Convert from cents to dollars
        Currency:  paymentSession.Currency,
    }

    payloadBytes, err := json.Marshal(paymentPayload)
    if err != nil {
        log.Printf("Failed to marshal payment session payload: %v", err)
        return err
    }

    if err := c.rabbitmq.PublishMessage(ctx, contracts.PaymentEventSessionCreated, contracts.AmqpMessage{
        OwnerID: payload.UserID,
        Data:    payloadBytes,
    }); err != nil {
        log.Printf("Failed to publish payment session created event: %v", err)
        return err
    }

    log.Printf("Published payment session created event for trip: %s", payload.TripID)
    return nil
}
```

FOLDER: services/payment-service/internal/infrastructure/stripe/**FILE: stripe.go**

```
package stripe

import (
    "context"
    "fmt"

    "ride-sharing/services/payment-service/internal/domain"
    "ride-sharing/services/payment-service/pkg/types"

    "github.com/stripe/stripe-go/v81"
    "github.com/stripe/stripe-go/v81/checkout/session"
)

type stripeClient struct {
    config *types.PaymentConfig
}

func NewStripeClient(config *types.PaymentConfig) domain.PaymentProcessor {
    stripe.Key = config.StripeSecretKey

    return &stripeClient{
        config: config,
    }
}

func (s *stripeClient) CreatePaymentSession(ctx context.Context, amount int64, currency string, metadata map[string]string) (string, error) {
    params := &stripe.CheckoutSessionParams{
        SuccessURL: stripe.String(s.config.SuccessURL),
        CancelURL:  stripe.String(s.config.CancelURL),
        Metadata:   metadata,
        LineItems: []*stripe.CheckoutSessionLineItemParams{
            {
                PriceData: &stripe.CheckoutSessionLineItemPriceDataParams{
                    Currency: stripe.String(currency),
                    ProductData: &stripe.CheckoutSessionLineItemPriceDataProductDataParams{
                        Name: stripe.String("Ride Payment"),
                    },
                    UnitAmount: stripe.Int64(amount),
                },
                Quantity: stripe.Int64(1),
            },
        },
        Mode: stripe.String(string(stripe.CheckoutSessionModePayment)),
    }

    result, err := session.New(params)
    if err != nil {
        return "", fmt.Errorf("failed to create a payment session on stripe: %w", err)
    }

    return result.ID, nil
}
```

FOLDER: services/payment-service/internal/service/**FILE: service.go**

```
package service

import (
    "context"
    "fmt"
    "time"

    "ride-sharing/services/payment-service/internal/domain"
)
```

```
"ride-sharing/services/payment-service/pkg/types"

"github.com/google/uuid"
)

type paymentService struct {
    paymentProcessor domain.PaymentProcessor
}

// NewPaymentService creates a new instance of the payment service
func NewPaymentService(paymentProcessor domain.PaymentProcessor) domain.Service {
    return &paymentService{
        paymentProcessor: paymentProcessor,
    }
}

// CreatePaymentSession creates a new payment session for a trip
func (s *paymentService) CreatePaymentSession(
    ctx context.Context,
    tripID string,
    userID string,
    driverID string,
    amount int64,
    currency string,
) (*types.PaymentIntent, error) {
    metadata := map[string]string{
        "trip_id": tripID,
        "user_id": userID,
        "driver_id": driverID,
    }

    sessionID, err := s.paymentProcessor.CreatePaymentSession(ctx, amount, currency, metadata)
    if err != nil {
        return nil, fmt.Errorf("failed to create payment session: %w", err)
    }

    paymentIntent := &types.PaymentIntent{
        ID:          uuid.New().String(),
        TripID:      tripID,
        UserID:      userID,
        DriverID:    driverID,
        Amount:      amount,
        Currency:    currency,
        StripeSessionID: sessionID,
        CreatedAt:   time.Now(),
    }

    return paymentIntent, nil
}
```

FOLDER: services/payment-service/pkg/types/**FILE: types.go**

```
package types

import "time"

// PaymentStatus represents the current status of a payment
type PaymentStatus string

const (
    PaymentStatusPending   PaymentStatus = "pending"
    PaymentStatusSuccess   PaymentStatus = "success"
    PaymentStatusFailed    PaymentStatus = "failed"
    PaymentStatusCancelled PaymentStatus = "cancelled"
)

// Payment represents a payment transaction
type Payment struct {
    ID          string    `json:"id"`
    TripID      string    `json:"trip_id"`
    UserID      string    `json:"user_id"`
    Amount      int64     `json:"amount"` // Amount in cents
    Currency    string    `json:"currency"` // e.g., "usd"
    Status      PaymentStatus `json:"status"`
    StripeSessionID string  `json:"stripe_session_id"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

// PaymentIntent represents the intent to collect a payment
type PaymentIntent struct {
    ID          string    `json:"id"`
    TripID      string    `json:"trip_id"`
    UserID      string    `json:"user_id"`
    DriverID    string    `json:"driver_id"`
    Amount      int64     `json:"amount"`
    Currency    string    `json:"currency"`
    StripeSessionID string  `json:"stripe_session_id"`
    CreatedAt   time.Time `json:"created_at"`
}

// PaymentConfig holds the configuration for the payment service
type PaymentConfig struct {
    StripeSecretKey string `json:"stripeSecretKey"`
    StripeWebhookSecret string `json:"stripeWebhookSecret"`
    Currency string `json:"currency"`
    SuccessURL string `json:"successURL"`
    CancelURL string `json:"cancelURL"`
}

// PaymentConfig holds the configuration for the payment service
type PaymentConfig struct {
    StripeSecretKey string `json:"stripeSecretKey"`
    StripeWebhookSecret string `json:"stripeWebhookSecret"`
    Currency string `json:"currency"`
    SuccessURL string `json:"successURL"`
    CancelURL string `json:"cancelURL"`
}
```

FOLDER: services/trip-service/cmd/**FILE: main.go**

```
package main

import (
    "context"
    "log"
    "net"
    "os"
    "os/signal"
    "ride-sharing/services/trip-service/internal/infrastructure/events"
    "ride-sharing/services/trip-service/internal/infrastructure/grpc"
    "ride-sharing/services/trip-service/internal/infrastructure/repository"
    "ride-sharing/services/trip-service/internal/service"
    "ride-sharing/shared/db"
    "ride-sharing/shared/env"
    "ride-sharing/shared/messaging"
    "ride-sharing/shared/tracing"
    "syscall"

    grpcserver "google.golang.org/grpc"
)

var GrpcAddr = ":9093"

func main() {
    // Initialize Tracing
```

```
    tracerCfg := tracing.Config{
        ServiceName: "trip-service",
        Environment: env.GetString("ENVIRONMENT", "development"),
        JaegerEndpoint: env.GetString("JAEGER_ENDPOINT", "http://jaeger:14268/api/traces"),
    }

    sh, err := tracing.InitTracer(tracerCfg)
    if err != nil {
        log.Fatalf("Failed to initialize the tracer: %v", err)
    }

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    defer sh(ctx)

    // Initialize MongoDB
    mongoClient, err := db.NewMongoClient(ctx, db.NewMongoDefaultConfig())
    if err != nil {
        log.Fatalf("Failed to initialize MongoDB, err: %v", err)
    }
    defer mongoClient.Disconnect(ctx)

    mongoDb := db.GetDatabase(mongoClient, db.NewMongoDefaultConfig())

    rabbitMqURI := env.GetString("RABBITMQ_URI", "amqp://guest:guest@rabbitmq:5672/")

    mongoDBRepo := repository.NewMongoRepository(mongoDb)
    svc := service.NewService(mongoDBRepo)

    go func() {
        sigCh := make(chan os.Signal, 1)
        signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
        <-sigCh
        cancel()
    }()

    lis, err := net.Listen("tcp", GrpcAddr)
    if err != nil {
        log.Fatalf("Failed to listen: %v", err)
    }

    // RabbitMQ connection
    rabbitmq, err := messaging.NewRabbitMQ(rabbitMqURI)
    if err != nil {
        log.Fatal(err)
    }
    defer rabbitmq.Close()

    log.Println("Starting RabbitMQ connection")

    publisher := events.NewTripEventPublisher(rabbitmq)

    // Start driver consumer
    driverConsumer := events.NewDriverConsumer(rabbitmq, svc)
    go driverConsumer.Listen()

    // Start payment consumer
    paymentConsumer := events.NewPaymentConsumer(rabbitmq, svc)
    go paymentConsumer.Listen()

    // Starting the gRPC server
    grpcServer := grpcserver.NewServer(tracing.WithTracingInterceptors(...))
    grpc.NewGRPCHandler(grpcServer, svc, publisher)

    log.Printf("Starting gRPC server Trip service on port %s", lis.Addr().String())

    go func() {
        if err := grpcServer.Serve(lis); err != nil {
            log.Printf("failed to serve: %v", err)
            cancel()
        }
    }()

    // wait for the shutdown signal
    <-ctx.Done()
    log.Println("Shutting down the server...")
    grpcServer.GracefulStop()
}
```

FOLDER: services/trip-service/internal/domain/**FILE: ride_fare.go**

```
package domain

import (
    "ride-sharing/services/trip-service/pkg/types"
    pb "ride-sharing/shared/proto/trip"

    "go.mongodb.org/mongo-driver/bson/primitive"
)

type RideFareModel struct {
    ID          primitive.ObjectID `bson:"_id,omitempty"`
    UserID      string              `bson:"userID"`
    PackageSlug string              `bson:"packageSlug"` // ex: van, luxury,
    sedan
    TotalPriceInCents float64 `bson:"totalPriceInCents"`
    Route              *types.OSRMApiResponse `bson:"route"`
}

func (r *RideFareModel) ToProto() *pb.RideFare {
    return &pb.RideFare{
        ID:          r.ID.Hex(),
        UserID:      r.UserID,
        PackageSlug: r.PackageSlug,
        TotalPriceInCents: r.TotalPriceInCents,
    }
}

func ToRideFaresProto(fares []*RideFareModel) []*pb.RideFare {
    var protoFares []*pb.RideFare
    for _, f := range fares {
        protoFares = append(protoFares, f.ToProto())
    }
    return protoFares
}

// FILE: trip.go
package domain

import (
    "context"
    "ride-sharing/shared/types"

    tripTypes "ride-sharing/services/trip-service/pkg/types"
    pb "ride-sharing/shared/proto/trip"
    "go.mongodb.org/mongo-driver/bson/primitive"
```

```

)

type TripModel struct {
    ID          primitive.ObjectID `bson:"_id,omitempty"`
    UserID      string          `bson:"userID"`
    Status      string          `bson:"status"`
    RideFare    *RideFareModel
    Driver      *pb.TripDriver `bson:"driver"`
}

func (t *TripModel) ToProto() *pb.Trip {
    return &pb.Trip{
        Id:      t.ID.Hex(),
        UserID:  t.UserID,
        SelectedFare: t.RideFare.ToProto(),
        Status:   t.Status,
        Driver:   t.Driver,
        Route:    t.RideFare.Route.ToProto(),
    }
}

type TripRepository interface {
    CreateTrip(ctx context.Context, trip *TripModel) (*TripModel, error)
    SaveRideFare(ctx context.Context, f *RideFareModel) error
    GetRideFareByID(ctx context.Context, id string) (*RideFareModel, error)
    GetTripByID(ctx context.Context, id string) (*TripModel, error)
    UpdateTrip(ctx context.Context, tripID string, status string, driver *pb.Driver) error
}

type TripService interface {
    CreateTrip(ctx context.Context, fare *RideFareModel) (*TripModel, error)
    GetRoute(ctx context.Context, pickup, destination *types.Coordinate, useOsrMapi bool) (*tripTypes.OsrMapiResponse, error)
    EstimatePackagesPriceWithRoute(route *tripTypes.OsrMapiResponse) []*RideFareModel
    GenerateTripFares(
        ctx context.Context,
        fares []*RideFareModel,
        userID string,
        route *tripTypes.OsrMapiResponse,
    ) ([]*RideFareModel, error)
    GetAndValidateFare(ctx context.Context, fareID, userID string) (*RideFareModel, error)
    GetTripByID(ctx context.Context, id string) (*TripModel, error)
    UpdateTrip(ctx context.Context, tripID string, status string, driver *pb.Driver) error
}

```

FOLDER: services/trip-service/internal/infrastructure/events/

FILE: driver_consumer.go

```

package events

import (
    "context"
    "encoding/json"
    "fmt"
    "log"

    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"
    pbd "ride-sharing/shared/proto/driver"

    "github.com/rabbitmq/amqp091-go"
)

type driverConsumer struct {
    rabbitmq *messaging.RabbitMQ
    service  domain.TripService
}

func NewDriverConsumer(rabbitmq *messaging.RabbitMQ, service domain.TripService) *driverConsumer {
    return &driverConsumer{
        rabbitmq: rabbitmq,
        service:  service,
    }
}

func (c *driverConsumer) Listen() error {
    return c.rabbitmq.ConsumeMessages(messaging.DriverTripResponseQueue, func(ctx context.Context, msg amqp091.Delivery) error {
        var message contracts.AmqpMessage
        if err := json.Unmarshal(msg.Body, &message); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        var payload messaging.DriverTripResponseData
        if err := json.Unmarshal(message.Data, &payload); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        log.Printf("driver response received message: %v", payload)

        switch msg.RoutingKey {
        case contracts.DriverCmdTripAccept:
            if err := c.handleTripAccepted(ctx, payload.TripID, payload.Driver); err != nil {
                log.Printf("Failed to handle the trip accept: %v", err)
                return err
            }
        case contracts.DriverCmdTripDecline:
            if err := c.handleTripDeclined(ctx, payload.TripID, payload.RiderID); err != nil {
                log.Printf("Failed to handle the trip decline: %v", err)
                return err
            }
        }
        log.Printf("unknown trip event: %v", payload)
        return nil
    })
}

func (c *driverConsumer) handleTripDeclined(ctx context.Context, tripID, riderID string) error {
    // When a driver declines, we should try to find another driver

    trip, err := c.service.GetTripByID(ctx, tripID)
    if err != nil {
        return err
    }

    newPayload := messaging.TripEventData{
        Trip: trip.ToProto(),
    }
}

```

```

marshalledPayload, err := json.Marshal(newPayload)
if err != nil {
    return err
}

if err := c.rabbitmq.PublishMessage(ctx, contracts.TripEventDriverNotInterested,
    contracts.AmqpMessage{
        OwnerID: riderID,
        Data:    marshalledPayload,
    },
); err != nil {
    return err
}

return nil
}

func (c *driverConsumer) handleTripAccepted(ctx context.Context, tripID string, driver *pbd.Driver) error {
    // 1. Fetch the first
    trip, err := c.service.GetTripByID(ctx, tripID)
    if err != nil {
        return err
    }

    if trip == nil {
        return fmt.Errorf("Trip was not found %s", tripID)
    }

    // 2. Update the trip
    if err := c.service.UpdateTrip(ctx, tripID, "accepted", driver); err != nil {
        log.Printf("Failed to update the trip: %v", err)
        return err
    }

    trip, err = c.service.GetTripByID(ctx, tripID)
    if err != nil {
        return err
    }

    // 3. Driver has been assigned -> publish this event to RB
    marshalledTrip, err := json.Marshal(trip)
    if err != nil {
        return err
    }

    // Notify the rider that a driver has been assigned
    if err := c.rabbitmq.PublishMessage(ctx, contracts.TripEventDriverAssigned, contracts.AmqpMessage{
        OwnerID: trip.UserID,
        Data:    marshalledTrip,
    }); err != nil {
        return err
    }

    marshalledPayload, err := json.Marshal(messaging.PaymentTripResponseData{
        TripID: tripID,
        UserID:  trip.UserID,
        DriverID: driver.ID,
        Amount:  trip.RideFare.TotalPriceInCents,
        Currency: "USD",
    })

    if err := c.rabbitmq.PublishMessage(ctx, contracts.PaymentCmdCreateSession,
        contracts.AmqpMessage{
            OwnerID: trip.UserID,
            Data:    marshalledPayload,
        },
    ); err != nil {
        return err
    }

    return nil
}

```

FILE: payment_consumer.go

```

package events

import (
    "context"
    "encoding/json"
    "log"

    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"

    "github.com/rabbitmq/amqp091-go"
)

type paymentConsumer struct {
    rabbitmq *messaging.RabbitMQ
    service  domain.TripService
}

func NewPaymentConsumer(rabbitmq *messaging.RabbitMQ, service domain.TripService) *paymentConsumer {
    return &paymentConsumer{
        rabbitmq: rabbitmq,
        service:  service,
    }
}

func (c *paymentConsumer) Listen() error {
    return c.rabbitmq.ConsumeMessages(messaging.NotifyPaymentSuccessQueue, func(ctx context.Context, msg amqp091.Delivery) error {
        var message contracts.AmqpMessage
        if err := json.Unmarshal(msg.Body, &message); err != nil {
            log.Printf("Failed to unmarshal message: %v", err)
            return err
        }

        var payload messaging.PaymentStatusUpdateData
        if err := json.Unmarshal(message.Data, &payload); err != nil {
            log.Printf("Failed to unmarshal payload: %v", err)
            return err
        }

        log.Printf("Trip has been completed and paid.")

        return c.service.UpdateTrip(
            ctx,
            payload.TripID,
            "paid",
            nil,
        )
    })
}

```

FILE: trip_publisher.go

```
package events

import (
    "context"
    "encoding/json"
    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/shared/contracts"
    "ride-sharing/shared/messaging"
)

type TripEventPublisher struct {
    rabbitmq *messaging.RabbitMQ
}

func NewTripEventPublisher(rabbitmq *messaging.RabbitMQ) *TripEventPublisher {
    return &TripEventPublisher{
        rabbitmq: rabbitmq,
    }
}

func (p *TripEventPublisher) PublishTripCreated(ctx context.Context, trip *domain.TripModel) error {
    payload := messaging.TripEventData{
        Trip: trip.ToProto(),
    }

    tripEventJSON, err := json.Marshal(payload)
    if err != nil {
        return err
    }

    return p.rabbitmq.PublishMessage(ctx, contracts.TripEventCreated, contracts.AmqpMessage{
        OwnerID: trip.UserID,
        Data:    tripEventJSON,
    })
}
```

FOLDER: services/trip-service/internal/infrastructure/grpc/**FILE: grpc_handler.go**

```
package grpc

import (
    "context"
    "log"
    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/services/trip-service/internal/infrastructure/events"
    pb "ride-sharing/shared/proto/trip"
    "ride-sharing/shared/types"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

type GRPCHandler struct {
    pb.UnimplementedTripServiceServer

    service domain.TripService
    publisher *events.TripEventPublisher
}

func NewGRPCHandler(server *grpc.Server, service domain.TripService, publisher *events.TripEventPublisher) *GRPCHandler {
    handler := &GRPCHandler{
        service: service,
        publisher: publisher,
    }

    pb.RegisterTripServiceServer(server, handler)
    return handler
}

func (h *GRPCHandler) CreateTrip(ctx context.Context, req *pb.CreateTripRequest) (*pb.CreateTripResponse, error) {
    fareID := req.GetRideFareID()
    userID := req.GetUserID()

    rideFare, err := h.service.GetAndValidateFare(ctx, fareID, userID)
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to validate the fare: %v", err)
    }

    trip, err := h.service.CreateTrip(ctx, rideFare)
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to create the trip: %v", err)
    }

    if err := h.publisher.PublishTripCreated(ctx, trip); err != nil {
        return nil, status.Errorf(codes.Internal, "failed to publish the trip created event: %v", err)
    }

    return &pb.CreateTripResponse{
        TripID: trip.ID.Hex(),
    }, nil
}

func (h *GRPCHandler) PreviewTrip(ctx context.Context, req *pb.PreviewTripRequest) (*pb.PreviewTripResponse, error) {
    pickup := req.GetStartLocation()
    destination := req.GetEndLocation()

    pickupCoord := &types.Coordinate{
        Latitude: pickup.Latitude,
        Longitude: pickup.Longitude,
    }
    destinationCoord := &types.Coordinate{
        Latitude: destination.Latitude,
        Longitude: destination.Longitude,
    }

    userID := req.GetUserID()

    // CHANGE THE LAST ARG TO "FALSE" if the OSRM API is not working right now
    route, err := h.service.GetRoute(ctx, pickupCoord, destinationCoord, true)
    if err != nil {
        log.Println(err)
        return nil, status.Errorf(codes.Internal, "failed to get route: %v", err)
    }

    estimatedFares := h.service.EstimatePackagesPriceWithRoute(route)

    fares, err := h.service.GenerateTripFares(ctx, estimatedFares, userID, route)
    if err != nil {

```

```
        return nil, status.Errorf(codes.Internal, "failed to generate the ride fares: %v", err)
    }

    return &pb.PreviewTripResponse{
        Route:    route.ToProto(),
        RideFares: domain.ToRideFaresProto(fares),
    }, nil
}
```

FOLDER: services/trip-service/internal/infrastructure/http/**FILE: http.go**

```
package http

import (
    "encoding/json"
    "log"
    "net/http"
    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/shared/types"
)

type HTTPHandler struct {
    Service domain.TripService
}

type previewTripRequest struct {
    UserID      string `json:"userID"`
    Pickup      types.Coordinate `json:"pickup"`
    Destination types.Coordinate `json:"destination"`
}

func (s *HTTPHandler) HandleTripPreview(w http.ResponseWriter, r *http.Request) {
    var reqBody previewTripRequest
    if err := json.NewDecoder(r.Body).Decode(&reqBody); err != nil {
        http.Error(w, "failed to parse JSON data", http.StatusBadRequest)
        return
    }

    ctx := r.Context()

    // CHANGE THE LAST ARG TO "FALSE" if the OSRM API is not working right now
    t, err := s.Service.GetRoute(ctx, &reqBody.Pickup, &reqBody.Destination, true)
    if err != nil {
        log.Println(err)
    }

    writeJSON(w, http.StatusOK, t)
}

func writeJSON(w http.ResponseWriter, status int, data any) error {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    return json.NewEncoder(w).Encode(data)
}
```

FOLDER: services/trip-service/internal/infrastructure/repository/**FILE: innmem.go**

```
package repository

import (
    "context"
    "fmt"
    "ride-sharing/services/trip-service/internal/domain"
    pb "ride-sharing/shared/proto/driver"
    "ride-sharing/shared/proto/trip"
)

type InnmemRepository struct {
    trips      map[string]*domain.TripModel
    rideFares map[string]*domain.RideFareModel
}

func NewInnmemRepository() *InnmemRepository {
    return &InnmemRepository{
        trips:      make(map[string]*domain.TripModel),
        rideFares: make(map[string]*domain.RideFareModel),
    }
}

func (r *InnmemRepository) GetTripByID(ctx context.Context, id string) (*domain.TripModel, error) {
    trip, ok := r.trips[id]
    if !ok {
        return nil, nil
    }
    return trip, nil
}

func (r *InnmemRepository) UpdateTrip(ctx context.Context, tripID string, status string, driver *pb.Driver) error {
    trip, ok := r.trips[tripID]
    if !ok {
        return fmt.Errorf("trip not found with ID: %s", tripID)
    }

    trip.Status = status

    if driver != nil {
        trip.Driver = &pb.TripDriver{
            ID:      driver.ID,
            Name:    driver.Name,
            CarPlate: driver.CarPlate,
            ProfilePicture: driver.ProfilePicture,
        }
    }
    return nil
}

func (r *InnmemRepository) GetRideFareByID(ctx context.Context, id string) (*domain.RideFareModel, error) {
    fare, exist := r.rideFares[id]
    if !exist {
        return nil, fmt.Errorf("fare does not exist with ID: %s", id)
    }

    return fare, nil
}

func (r *InnmemRepository) CreateTrip(ctx context.Context, trip *domain.TripModel) (*domain.TripModel, error) {
    r.trips[trip.ID.Hex()] = trip
    return trip, nil
}

func (r *InnmemRepository) SaveRideFare(ctx context.Context, f *domain.RideFareModel) error {
    r.rideFares[f.ID.Hex()] = f
}
```



```

    return nil
}
FILE: mongodb.go
package repository

import (
    "context"
    "fmt"

    "ride-sharing/services/trip-service/internal/domain"
    "ride-sharing/shared/db"
    pbd "ride-sharing/shared/proto/driver"

    "go.mongodb.org/mongo-driver/bson"
    "go.mongodb.org/mongo-driver/bson/primitive"
    "go.mongodb.org/mongo-driver/mongo"
)

type mongoRepository struct {
    db *mongo.Database
}

func NewMongoRepository(db *mongo.Database) *mongoRepository {
    return &mongoRepository{db: db}
}

func (r *mongoRepository) CreateTrip(ctx context.Context, trip *domain.TripModel) (*domain.TripModel, error) {
    result, err := r.db.Collection(db.TripsCollection).InsertOne(ctx, trip)
    if err != nil {
        return nil, err
    }

    trip.ID = result.InsertedID.(primitive.ObjectID)

    return trip, nil
}

func (r *mongoRepository) GetTripByID(ctx context.Context, id string) (*domain.TripModel, error) {
    _id, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return nil, err
    }

    result := r.db.Collection(db.TripsCollection).FindOne(ctx, bson.M{"_id": _id})
    if result.Err() != nil {
        return nil, result.Err()
    }

    var trip domain.TripModel
    err = result.Decode(&trip)
    if err != nil {
        return nil, err
    }

    return &trip, nil
}

func (r *mongoRepository) UpdateTrip(ctx context.Context, tripID string, status string, driver *pbd.Driver) error {
    _id, err := primitive.ObjectIDFromHex(tripID)
    if err != nil {
        return err
    }

    update := bson.M{"$set": bson.M{"status": status}}

    if driver != nil {
        update["$set"].(bson.M)["driver"] = driver
    }

    result, err := r.db.Collection(db.TripsCollection).UpdateOne(ctx, bson.M{"_id": _id}, update)
    if err != nil {
        return err
    }

    if result.ModifiedCount == 0 {
        return fmt.Errorf("trip not found: %s", tripID)
    }

    return nil
}

func (r *mongoRepository) SaveRideFare(ctx context.Context, fare *domain.RideFareModel) error {
    result, err := r.db.Collection(db.RideFaresCollection).InsertOne(ctx, fare)
    if err != nil {
        return err
    }

    fare.ID = result.InsertedID.(primitive.ObjectID)

    return nil
}

func (r *mongoRepository) GetRideFareByID(ctx context.Context, id string) (*domain.RideFareModel, error) {
    _id, err := primitive.ObjectIDFromHex(id)
    if err != nil {
        return nil, err
    }

    result := r.db.Collection(db.RideFaresCollection).FindOne(ctx, bson.M{"_id": _id})
    if result.Err() != nil {
        return nil, result.Err()
    }

    var fare domain.RideFareModel
    err = result.Decode(&fare)
    if err != nil {
        return nil, err
    }

    return &fare, nil
}

```

FOLDER: services/trip-service/internal/service/

FILE: service.go

package service

```

import (
    "context"
    "encoding/json"
    "fmt"
    "io"
    "log"

```

```

    "net/http"
    "ride-sharing/services/trip-service/internal/domain"
    tripTypes "ride-sharing/services/trip-service/pkg/types"
    "ride-sharing/shared/env"
    pbd "ride-sharing/shared/proto/driver"
    "ride-sharing/shared/proto/trip"
    "ride-sharing/shared/types"

    "go.mongodb.org/mongo-driver/bson/primitive"
)

type service struct {
    repo domain.TripRepository
}

func NewService(repo domain.TripRepository) *service {
    return &service{
        repo: repo,
    }
}

func (s *service) CreateTrip(ctx context.Context, fare *domain.RideFareModel) (*domain.TripModel, error) {
    t := &domain.TripModel{
        ID:      primitive.NewObjectID(),
        UserID:  fare.UserID,
        Status:  "pending",
        RideFare: fare,
        Driver:  &trip.TripDriver{},
    }

    return s.repo.CreateTrip(ctx, t)
}

func (s *service) GetRoute(ctx context.Context, pickup, destination *types.Coordinate, useOSRM bool) (*tripTypes.OSRMApiResponse, error) {
    if !useOSRM {
        // Return a simple mock response in case we don't want to rely on an external API
        return &tripTypes.OSRMApiResponse{
            Routes: []struct {
                Distance float64 `json:"distance"`
                Duration float64 `json:"duration"`
                Geometry struct {
                    Coordinates [][]float64 `json:"coordinates"`
                } `json:"geometry"`
            }{
                {
                    Distance: 5.0, // 5km
                    Duration: 600, // 10 minutes
                    Geometry: struct {
                        Coordinates [][]float64 `json:"coordinates"`
                    }{
                        Coordinates: [][]float64{
                            {pickup.Latitude, pickup.Longitude},
                            {destination.Latitude, destination.Longitude},
                        },
                    },
                },
            }, nil
        }

        // or use our self hosted API (check the course lesson: "Preparing for External API Failures")
        baseURL := env.GetString("OSRM_API", "http://router.project-osrm.org")

        url := fmt.Sprintf(
            "%s/route/v1/driving/%f,%f;%f,%f?overview=full&geometries=geojson",
            baseURL,
            pickup.Longitude, pickup.Latitude,
            destination.Longitude, destination.Latitude,
        )

        log.Printf("Started Fetching from OSRM API: URL: %s", url)

        resp, err := http.Get(url)
        if err != nil {
            return nil, fmt.Errorf("failed to fetch route from OSRM API: %v", err)
        }
        defer resp.Body.Close()

        body, err := io.ReadAll(resp.Body)
        if err != nil {
            return nil, fmt.Errorf("failed to read the response: %v", err)
        }

        log.Printf("Got response from OSRM API %s", string(body))

        var routeResp tripTypes.OSRMApiResponse
        if err := json.Unmarshal(body, &routeResp); err != nil {
            return nil, fmt.Errorf("failed to parse response: %v", err)
        }

        return &routeResp, nil
    }

    func (s *service) EstimatePackagePriceWithRoute(route *tripTypes.OSRMApiResponse) []*domain.RideFareModel {
        baseFares := getBaseFares()
        estimatedFares := make([]*domain.RideFareModel, len(baseFares))

        for i, f := range baseFares {
            estimatedFares[i] = estimateFareRoute(f, route)
        }

        return estimatedFares
    }

    func (s *service) GenerateTripFares(ctx context.Context, rideFares []*domain.RideFareModel, userID string, route *tripTypes.OSRMApiResponse) []*domain.RideFareModel {
        fares := make([]*domain.RideFareModel, len(rideFares))

        for i, f := range rideFares {
            id := primitive.NewObjectID()

            fare := &domain.RideFareModel{
                UserID:  userID,
                ID:      id,
                TotalPriceInCents: f.TotalPriceInCents,
                PackageSlug: f.PackageSlug,
                Route:    route,
            }

            if err := s.repo.SaveRideFare(ctx, fare); err != nil {
                return nil, fmt.Errorf("failed to save trip fare: %w", err)
            }
        }
    }

```

```
        fares[i] = fare
    }

    return fares, nil
}

func (s *service) GetAndValidateFare(ctx context.Context, fareID, userID string) (*domain.RideFareModel, error) {
    fare, err := s.repo.GetRideFareByID(ctx, fareID)
    if err != nil {
        return nil, fmt.Errorf("failed to get trip fare: %w", err)
    }

    if fare == nil {
        return nil, fmt.Errorf("fare does not exist")
    }

    // User fare validation (user is owner of this fare?)
    if userID != fare.UserID {
        return nil, fmt.Errorf("fare does not belong to the user")
    }

    return fare, nil
}

func estimateFareRoute(f *domain.RideFareModel, route *tripTypes.OsrmApiResponse) *domain.RideFareModel {
    pricingCfg := tripTypes.DefaultPricingConfig()
    carPackagePrice := f.TotalPriceInCents

    distanceKm := route.Routes[0].Distance
    durationInMinutes := route.Routes[0].Duration

    distanceFare := distanceKm * pricingCfg.PricePerUnitOfDistance
    timeFare := durationInMinutes * pricingCfg.PricingPerMinute
    totalPrice := carPackagePrice + distanceFare + timeFare

    return &domain.RideFareModel{
        TotalPriceInCents: totalPrice,
        PackageSlug:       f.PackageSlug,
    }
}

func getBaseFares() []*domain.RideFareModel {
    return []*domain.RideFareModel{
        {
            PackageSlug: "suv",
            TotalPriceInCents: 200,
        },
        {
            PackageSlug: "sedan",
            TotalPriceInCents: 350,
        },
        {
            PackageSlug: "van",
            TotalPriceInCents: 400,
        },
        {
            PackageSlug: "luxury",
            TotalPriceInCents: 1000,
        },
    }
}

func (s *service) GetTripByID(ctx context.Context, id string) (*domain.TripModel, error) {
    return s.repo.GetTripByID(ctx, id)
}

func (s *service) UpdateTrip(ctx context.Context, tripID string, status string, driver *pb.Driver) error {
    return s.repo.UpdateTrip(ctx, tripID, status, driver)
}
```

FOLDER: services/trip-service/pkg/types/**FILE: types.go**

```
package types

import pb "ride-sharing/shared/proto/trip"

type OsrmApiResponse struct {
    Routes []struct {
        Distance float64 `json:"distance"`
        Duration float64 `json:"duration"`
        Geometry struct {
            Coordinates [][]float64 `json:"coordinates"`
        } `json:"geometry"`
    } `json:"routes"`
}

func (o *OsrmApiResponse) ToProto() *pb.Route {
    route := o.Routes[0]
    geometry := route.Geometry.Coordinates
    coordinates := make([]*pb.Coordinate, len(geometry))
    for i, coord := range geometry {
        coordinates[i] = &pb.Coordinate{
            Latitude: coord[0],
            Longitude: coord[1],
        }
    }

    return &pb.Route{
        Geometry: []*pb.Geometry{
            {
                Coordinates: coordinates,
            },
        },
        Distance: route.Distance,
        Duration: route.Duration,
    }
}

type PricingConfig struct {
    PricePerUnitOfDistance float64
    PricingPerMinute        float64
}

func DefaultPricingConfig() *PricingConfig {
    return &PricingConfig{
        PricePerUnitOfDistance: 1.5,
        PricingPerMinute:       0.25,
    }
}
```

FOLDER: shared/contracts/**FILE: amqp.go**

```
package contracts
```

```
// AmqpMessage is the message structure for AMQP.
type AmqpMessage struct {
    OwnerID string `json:"ownerId"`
    Data    []byte  `json:"data"`
}

// Routing keys - using consistent event/command patterns
const (
    // Trip events (trip.event.*)
    TripEventCreated      = "trip.event.created"
    TripEventDriverAssigned = "trip.event.driver_assigned"
    TripEventNoDriversFound = "trip.event.no_drivers_found"
    TripEventDriverNotInterested = "trip.event.driver_not_interested"

    // Driver commands (driver.cmd.*)
    DriverCmdTripRequest = "driver.cmd.trip_request"
    DriverCmdTripAccept = "driver.cmd.trip_accept"
    DriverCmdTripDecline = "driver.cmd.trip_decline"
    DriverCmdLocation = "driver.cmd.location"
    DriverCmdRegister = "driver.cmd.register"

    // Payment events (payment.event.*)
    PaymentEventSessionCreated = "payment.event.session_created"
    PaymentEventSuccess = "payment.event.success"
    PaymentEventFailed = "payment.event.failed"
    PaymentEventCancelled = "payment.event.cancelled"

    // Payment commands (payment.cmd.*)
    PaymentCmdCreateSession = "payment.cmd.create_session"
)
```

FILE: http.go

```
package contracts
```

```
// APIResponse is the response structure for the API.
type APIResponse struct {
    Data any `json:"data,omitempty"`
    Error *APIError `json:"error,omitempty"`
}

// APIError is the error structure for the API.
type APIError struct {
    Code string `json:"code"`
    Message string `json:"message"`
}
```

FILE: ws.go

```
package contracts
```

```
import "encoding/json"
```

```
// WSMessage is the message structure for the WebSocket.
type WSMessage struct {
    Type string `json:"type"`
    Data any `json:"data"`
}

type WSDriverMessage struct {
    Type string `json:"type"`
    Data json.RawMessage `json:"data"`
}
```

FOLDER: shared/db/**FILE: mongodb.go**

```
package db
```

```
import (
    "context"
    "fmt"
    "log"
    "os"
    "time"

    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
    "go.mongodb.org/mongo-driver/mongo/readpref"
)

const (
    TripsCollection = "trips"
    RideFaresCollection = "ride_fares"
)

// MongoConfig holds MongoDB connection configuration
type MongoConfig struct {
    URI string
    Database string
}

// NewMongoConfig creates a new MongoDB configuration from environment variables
func NewMongoDefaultConfig() *MongoConfig {
    return &MongoConfig{
        URI: os.Getenv("MONGODB_URI"),
        Database: "ride-sharing",
    }
}

// NewMongoClient creates a new MongoDB client
func NewMongoClient(ctx context.Context, cfg *MongoConfig) (*mongo.Client, error) {
    if cfg.URI == "" {
        return nil, fmt.Errorf("mongodb URI is required")
    }

    if cfg.Database == "" {
        return nil, fmt.Errorf("mongodb database is required")
    }

    ctx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
    defer cancel()
    client, err := mongo.Connect(ctx, options.Client().ApplyURI(cfg.URI))
    if err != nil {
        return nil, err
    }

    err = client.Ping(ctx, readpref.Primary())
    if err != nil {
        return nil, err
    }

    log.Printf("Successfully connected to MongoDB at %s", cfg.URI)
    return client, nil
}

// GetDatabase returns a database instance
func GetDatabase(client *mongo.Client, cfg *MongoConfig) *mongo.Database {
    return client.Database(cfg.Database)
}
```

FOLDER: shared/env/**FILE: env.go**

```

/*
Package env provides a simple way to get environment variables.
*/
package env

import (
    "os"
    "strconv"
)

func GetString(key, fallback string) string {
    val, ok := os.LookupEnv(key)
    if !ok {
        return fallback
    }
    return val
}

func GetInt(key string, fallback int) int {
    val, ok := os.LookupEnv(key)
    if !ok {
        return fallback
    }

    valAsInt, err := strconv.Atoi(val)
    if err != nil {
        return fallback
    }
    return valAsInt
}

func GetBool(key string, fallback bool) bool {
    val, ok := os.LookupEnv(key)
    if !ok {
        return fallback
    }

    boolVal, err := strconv.ParseBool(val)
    if err != nil {
        return fallback
    }
    return boolVal
}

```

FOLDER: shared/messaging/**FILE: connection_manager.go**

```

package messaging

import (
    "errors"
    "log"
    "net/http"
    "sync"

    "ride-sharing/shared/contracts"
    "github.com/gorilla/websocket"
)

var (
    ErrConnectionNotFound = errors.New("connection not found")
)

// connWrapper is a wrapper around the websocket connection to allow for thread-safe o
perations
// This is necessary because the websocket connection is not thread-safe
type connWrapper struct {
    conn *websocket.Conn
    mutex sync.Mutex
}

type ConnectionManager struct {
    connections map[string]*connWrapper // Local connections storage (userId -> connec
tion)
    mutex       sync.RWMutex
}

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool {
        return true // Allow all origins for now
    },
}

// Note that on multiple instances of the API gateway, the connection manager needs to
store the connections on a separate shared storage.
func NewConnectionManager() *ConnectionManager {
    return &ConnectionManager{
        connections: make(map[string]*connWrapper),
    }
}

func (cm *ConnectionManager) Upgrade(w http.ResponseWriter, r *http.Request) (*websock
et.Conn, error) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        return nil, err
    }
    return conn, nil
}

func (cm *ConnectionManager) Add(id string, conn *websocket.Conn) {
    cm.mutex.Lock()
    defer cm.mutex.Unlock()
    cm.connections[id] = &connWrapper{
        conn: conn,
        mutex: sync.Mutex{},
    }
    log.Printf("Added connection for user %s", id)
}

func (cm *ConnectionManager) Remove(id string) {
    cm.mutex.Lock()
    defer cm.mutex.Unlock()
    delete(cm.connections, id)
}

func (cm *ConnectionManager) Get(id string) (*websocket.Conn, bool) {
    cm.mutex.RLock()
    defer cm.mutex.RUnlock()
    wrapper, exists := cm.connections[id]
    if !exists {

```

```

        return nil, false
    }
    return wrapper.conn, true
}

func (cm *ConnectionManager) SendMessage(id string, message contracts.WSMessage) error
{
    cm.mutex.RLock()
    wrapper, exists := cm.connections[id]
    cm.mutex.RUnlock()

    if !exists {
        return ErrConnectionNotFound
    }

    wrapper.mutex.Lock()
    defer wrapper.mutex.Unlock()

    return wrapper.conn.WriteJSON(message)
}

```

FILE: events.go

```

package messaging

import (
    pbd "ride-sharing/shared/proto/driver"
    pb "ride-sharing/shared/proto/trip"
)

const (
    FindAvailableDriversQueue      = "find_available_drivers"
    DriverCmdTripRequestQueue     = "driver_cmd_trip_request"
    DriverTripResponseQueue       = "driver_trip_response"
    NotifyDriverNoDriversFoundQueue = "notify_driver_no_drivers_found"
    NotifyDriverAssignQueue       = "notify_driver_assign"
    PaymentTripResponseQueue      = "payment_trip_response"
    NotifyPaymentSessionCreatedQueue = "notify_payment_session_created"
    NotifyPaymentSuccessQueue     = "payment_success"
    DeadLetterQueue               = "dead_letter_queue"
)

type TripEventData struct {
    Trip *pb.Trip `json:"trip"`
}

type DriverTripResponseData struct {
    Driver  *pbd.Driver `json:"driver"`
    TripID  string      `json:"tripID"`
    RiderID string      `json:"riderID"`
}

type PaymentEventSessionCreatedData struct {
    TripID      string `json:"tripID"`
    SessionID   string `json:"sessionID"`
    Amount      float64 `json:"amount"`
    Currency    string `json:"currency"`
}

type PaymentTripResponseData struct {
    TripID      string `json:"tripID"`
    UserID      string `json:"userID"`
    DriverID    string `json:"driverID"`
    Amount      float64 `json:"amount"`
    Currency    string `json:"currency"`
}

type PaymentStatusUpdateData struct {
    TripID      string `json:"tripID"`
    UserID      string `json:"userID"`
    DriverID    string `json:"driverID"`
}

```

FILE: queue_consumer.go

```

package messaging

import (
    "encoding/json"
    "log"

    "ride-sharing/shared/contracts"
)

type QueueConsumer struct {
    rb *RabbitMQ
    connMgr *ConnectionManager
    queueName string
}

func NewQueueConsumer(rb *RabbitMQ, connMgr *ConnectionManager, queueName string) *Que
ueConsumer {
    return &QueueConsumer{
        rb: rb,
        connMgr: connMgr,
        queueName: queueName,
    }
}

func (qc *QueueConsumer) Start() error {
    msgs, err := qc.rb.Channel.Consume(
        qc.queueName,
        "",
        true,
        false,
        false,
        nil,
    )
    if err != nil {
        return err
    }

    go func() {
        for msg := range msgs {
            var msgBody contracts.AmqpMessage
            if err := json.Unmarshal(msg.Body, &msgBody); err != nil {
                log.Println("Failed to unmarshal message:", err)
                continue
            }

            userID := msgBody.OwnerID

            var payload any
            if msgBody.Data != nil {
                if err := json.Unmarshal(msgBody.Data, &payload); err != nil {
                    log.Println("Failed to unmarshal payload:", err)
                    continue
                }
            }

            clientMsg := contracts.WSMessage{

```

Page 12

```

        NotifyDriverNoDriversFoundQueue,
        []string{contracts.TripEventNoDriversFound},
        TripExchange,
    ); err != nil {
        return err
    }

    if err := r.declareAndBindQueue(
        NotifyDriverAssignQueue,
        []string{contracts.TripEventDriverAssigned},
        TripExchange,
    ); err != nil {
        return err
    }

    if err := r.declareAndBindQueue(
        PaymentTripResponseQueue,
        []string{contracts.PaymentCmdCreateSession},
        TripExchange,
    ); err != nil {
        return err
    }

    if err := r.declareAndBindQueue(
        NotifyPaymentSessionCreatedQueue,
        []string{contracts.PaymentEventSessionCreated},
        TripExchange,
    ); err != nil {
        return err
    }

    if err := r.declareAndBindQueue(
        NotifyPaymentSuccessQueue,
        []string{contracts.PaymentEventSuccess},
        TripExchange,
    ); err != nil {
        return err
    }

    return nil
}

func (r *RabbitMQ) declareAndBindQueue(queueName string, messageTypes []string, exchange string) error {
    // Add dead letter configuration
    args := amqp.Table{
        "x-dead-letter-exchange": DeadLetterExchange,
    }

    q, err := r.Channel.QueueDeclare(
        queueName, // name
        true,      // durable
        false,     // delete when unused
        false,     // exclusive
        false,     // no-wait
        args,      // arguments with DLX config
    )
    if err != nil {
        log.Fatal(err)
    }

    for _, msg := range messageTypes {
        if err := r.Channel.QueueBind(
            q.Name, // queue name
            msg,   // routing key
            exchange, // exchange
            false,
            nil,
        ); err != nil {
            return fmt.Errorf("failed to bind queue to %s: %v", queueName, err)
        }
    }

    return nil
}

func (r *RabbitMQ) Close() {
    if r.conn != nil {
        r.conn.Close()
    }
    if r.Channel != nil {
        r.Channel.Close()
    }
}

```

FOLDER: shared/proto/driver/

FILE: driver.pb.go

// Code generated by protoc-gen-go. DO NOT EDIT.

// versions:

// protoc-gen-go v1.36.6

// protoc v5.29.3

// source: driver.proto

```

package driver

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
    sync "sync"
    unsafe "unsafe"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type RegisterDriverRequest struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    DriverID       string                  `protobuf:"bytes,1,opt,name=driverID,proto3" json:"driverID,omitempty"`
    PackageSlug    string                  `protobuf:"bytes,2,opt,name=packageSlug,proto3" json:"packageSlug,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *RegisterDriverRequest) Reset() {
    *x = RegisterDriverRequest{}
    mi := &file_driver_proto_msgTypes[0]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *RegisterDriverRequest) String() string {

```

```

    return protoimpl.X.MessageStringOf(x)
}

func (*RegisterDriverRequest) ProtoMessage() {}

func (x *RegisterDriverRequest) ProtoReflect() protoreflect.Message {
    mi := &file_driver_proto_msgTypes[0]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use RegisterDriverRequest.ProtoReflect.Descriptor instead.
func (*RegisterDriverRequest) Descriptor() ([]byte, []int) {
    return file_driver_proto_rawDescGZIP(), []int{0}
}

func (x *RegisterDriverRequest) GetDriverID() string {
    if x != nil {
        return x.DriverID
    }
    return ""
}

func (x *RegisterDriverRequest) GetPackageSlug() string {
    if x != nil {
        return x.PackageSlug
    }
    return ""
}

type RegisterDriverResponse struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    Driver         *Driver                 `protobuf:"bytes,1,opt,name=driver,proto3" json:"driver,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *RegisterDriverResponse) Reset() {
    *x = RegisterDriverResponse{}
    mi := &file_driver_proto_msgTypes[1]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *RegisterDriverResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*RegisterDriverResponse) ProtoMessage() {}

func (x *RegisterDriverResponse) ProtoReflect() protoreflect.Message {
    mi := &file_driver_proto_msgTypes[1]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use RegisterDriverResponse.ProtoReflect.Descriptor instead.
func (*RegisterDriverResponse) Descriptor() ([]byte, []int) {
    return file_driver_proto_rawDescGZIP(), []int{1}
}

func (x *RegisterDriverResponse) GetDriver() *Driver {
    if x != nil {
        return x.Driver
    }
    return nil
}

type Driver struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    Id             string                  `protobuf:"bytes,1,opt,name=id,proto3" json:"id,omitempty"`
    Name           string                  `protobuf:"bytes,2,opt,name=name,proto3" json:"name,omitempty"`
    ProfilePicture string                  `protobuf:"bytes,3,opt,name=profilePicture,proto3" json:"profilePicture,omitempty"`
    CarPlate       string                  `protobuf:"bytes,4,opt,name=carPlate,proto3" json:"carPlate,omitempty"`
    Geohash        string                  `protobuf:"bytes,5,opt,name=geohash,proto3" json:"geohash,omitempty"`
    PackageSlug    string                  `protobuf:"bytes,6,opt,name=packageSlug,proto3" json:"packageSlug,omitempty"`
    Location       *Location               `protobuf:"bytes,7,opt,name=location,proto3" json:"location,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *Driver) Reset() {
    *x = Driver{}
    mi := &file_driver_proto_msgTypes[2]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Driver) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Driver) ProtoMessage() {}

func (x *Driver) ProtoReflect() protoreflect.Message {
    mi := &file_driver_proto_msgTypes[2]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Driver.ProtoReflect.Descriptor instead.
func (*Driver) Descriptor() ([]byte, []int) {
    return file_driver_proto_rawDescGZIP(), []int{2}
}

```

```

}

func (x *Driver) GetId() string {
    if x != nil {
        return x.Id
    }
    return ""
}

func (x *Driver) GetName() string {
    if x != nil {
        return x.Name
    }
    return ""
}

func (x *Driver) GetProfilePicture() string {
    if x != nil {
        return x.ProfilePicture
    }
    return ""
}

func (x *Driver) GetCarPlate() string {
    if x != nil {
        return x.CarPlate
    }
    return ""
}

func (x *Driver) GetGeohash() string {
    if x != nil {
        return x.Geohash
    }
    return ""
}

func (x *Driver) GetPackageSlug() string {
    if x != nil {
        return x.PackageSlug
    }
    return ""
}

func (x *Driver) GetLocation() *Location {
    if x != nil {
        return x.Location
    }
    return nil
}

type Location struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    Latitude   float64                 `protobuf:"fixed64,1,opt,name=latitude,proto3"`
    json:"latitude,omitempty"
    Longitude  float64                 `protobuf:"fixed64,2,opt,name=longitude,proto3"`
    json:"longitude,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

func (x *Location) Reset() {
    *x = Location{}
    mi := &file_driver_proto_msgTypes[3]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Location) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Location) ProtoMessage() {}

func (x *Location) ProtoReflect() protoreflect.Message {
    mi := &file_driver_proto_msgTypes[3]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Location.ProtoReflect.Descriptor instead.
func (*Location) Descriptor() ([]byte, []int) {
    return file_driver_proto_rawDescGZIP(), []int{3}
}

func (x *Location) GetLatitude() float64 {
    if x != nil {
        return x.Latitude
    }
    return 0
}

func (x *Location) GetLongitude() float64 {
    if x != nil {
        return x.Longitude
    }
    return 0
}

var File_driver_proto protoreflect.FileDescriptor

const file_driver_proto_rawDesc = "" +
    "\n" +
    "\tfdriver.proto\x12\x06driver\U0000" +
    "\x15RegisterDriverRequest\x12\x1a\n" +
    "\tdriverID\x18\x01 \x01(\tr\bdriverID\x12 \n" +
    "\vpackageSlug\x18\x02 \x01(\tr\vpackageSlug"@ \n" +
    "\x16RegisterDriverResponse\x12&\n" +
    "\x06driver\x18\x01 \x01(\v2\x0e.driver.DriverR\x06driver"\xda\x01\n" +
    "\x06driver\x12\x0e\n" +
    "\x02id\x18\x01 \x01(\tr\x02id\x12\x12\n" +
    "\x04name\x18\x02 \x01(\tr\x04name\x12&\n" +
    "\x0eProfilePicture\x18\x03 \x01(\tr\x0eProfilePicture\x12\x1a\n" +
    "\bcarPlate\x18\x04 \x01(\tr\bcarPlate\x12\x18\n" +
    "\ageohash\x18\x05 \x01(\tr\ageohash\x12 \n" +
    "\vpackageSlug\x18\x06 \x01(\tr\vpackageSlug\x12,\n" +
    "\blocation\x18\a \x01(\v2\x10.driver.LocationR\blocation"D\n" +
    "\bLocation\x12\x1a\n" +
    "\blatitude\x18\x01 \x01(\x01Rblatitude\x12\x1c\n" +
    "\tlongitude\x18\x02 \x01(\x01R\tlongitude2\xb3\x01\n" +
    "\rDriverService\x120\n" +
    "\x0eRegisterDriver\x12\x1d.driver.RegisterDriverRequest\x1a\x1e.driver.RegisterDriverResponse\x12Q\n" +
    "\x10UnregisterDriver\x12\x1d.driver.RegisterDriverRequest\x1a\x1e.driver.RegisterDriverResponse\x1cZ\x1aIshared/proto/driver;driverb\x06proto3"

var (
    file_driver_proto_rawDescOnce sync.Once
    file_driver_proto_rawDescData []byte
)

func file_driver_proto_rawDescGZIP() []byte {
    file_driver_proto_rawDescOnce.Do(func() {
        file_driver_proto_rawDescData = protoimpl.X.CompressGZIP(unsafe.Slice(unsafe.StringData(file_driver_proto_rawDesc), len(file_driver_proto_rawDesc)))
    })
    return file_driver_proto_rawDescData
}

var file_driver_proto_msgTypes = make([]protoimpl.MessageInfo, 4)
var file_driver_proto_goTypes = []any{
    (*RegisterDriverRequest)(nil), // 0: driver.RegisterDriverRequest
    (*RegisterDriverResponse)(nil), // 1: driver.RegisterDriverResponse
    (*Driver)(nil), // 2: driver.Driver
    (*Location)(nil), // 3: driver.Location
}
var file_driver_proto_depIdxs = []int32{
    2, // 0: driver.RegisterDriverResponse.driver:type_name -> driver.Driver
    3, // 1: driver.Driver.location:type_name -> driver.Location
    0, // 2: driver.DriverService.RegisterDriver:input_type -> driver.RegisterDriverRequest
    0, // 3: driver.DriverService.UnregisterDriver:input_type -> driver.RegisterDriverRequest
    1, // 4: driver.DriverService.RegisterDriver:output_type -> driver.RegisterDriverResponse
    1, // 5: driver.DriverService.UnregisterDriver:output_type -> driver.RegisterDriverResponse
    4, // [4:6] is the sub-list for method output_type
    2, // [2:4] is the sub-list for method input_type
    2, // [2:2] is the sub-list for extension type_name
    2, // [2:2] is the sub-list for extension extendee
    0, // [0:2] is the sub-list for field type_name
}

func init() { file_driver_proto_init() }
func file_driver_proto_init() {
    if File_driver_proto != nil {
        return
    }
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: unsafe.Slice(unsafe.StringData(file_driver_proto_rawDesc), len(file_driver_proto_rawDesc)),
            NumEnums: 0,
            NumMessages: 4,
            NumExtensions: 0,
            NumServices: 1,
        },
        GoTypes: file_driver_proto_goTypes,
        DependencyIndexes: file_driver_proto_depIdxs,
        MessageInfos: file_driver_proto_msgTypes,
    }.Build()
    File_driver_proto = out.File
    file_driver_proto_goTypes = nil
    file_driver_proto_depIdxs = nil
}

FILE: driver_grpc.pb.go
// Code generated by protoc-gen-go-grpc. DO NOT EDIT.
// versions:
// - protoc-gen-go-grpc v1.5.1
// - protoc v5.29.3
// source: driver.proto

package driver

import (
    context "context"
    grpc "google.golang.org/grpc"
    codes "google.golang.org/grpc/codes"
    status "google.golang.org/grpc/status"
)

// This is a compile-time assertion to ensure that this generated file
// is compatible with the grpc package it is being compiled against.
// Requires GRPC-GO v1.64.0 or later.
const _ = grpc.SupportPackageIsVersion9

const (
    DriverService_RegisterDriver_FullMethodName = "/driver.DriverService/RegisterDriver"
    DriverService_UnregisterDriver_FullMethodName = "/driver.DriverService/UnregisterDriver"
)

// DriverServiceClient is the client API for DriverService service.
//
// For semantics around ctx use and closing/ending streaming RPCs, please refer to https://pkg.go.dev/google.golang.org/grpc/?tab=doc#ClientConn.NewStream.
type DriverServiceClient interface {
    RegisterDriver(ctx context.Context, in *RegisterDriverRequest, opts ...grpc.CallOption) (*RegisterDriverResponse, error)
    UnregisterDriver(ctx context.Context, in *RegisterDriverRequest, opts ...grpc.CallOption) (*RegisterDriverResponse, error)
}

type driverServiceClient struct {
    cc grpc.ClientConnInterface
}

func NewDriverServiceClient(cc grpc.ClientConnInterface) DriverServiceClient {
    return &driverServiceClient{cc}
}

func (c *driverServiceClient) RegisterDriver(ctx context.Context, in *RegisterDriverRequest, opts ...grpc.CallOption) (*RegisterDriverResponse, error) {
    cOpts := append([]grpc.CallOption{grpc.StaticMethod()}, opts...)
    out := new(RegisterDriverResponse)
    err := c.cc.Invoke(ctx, DriverService_RegisterDriver_FullMethodName, in, out, cOpts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

func (c *driverServiceClient) UnregisterDriver(ctx context.Context, in *RegisterDriverRequest, opts ...grpc.CallOption) (*RegisterDriverResponse, error) {
    cOpts := append([]grpc.CallOption{grpc.StaticMethod()}, opts...)
    out := new(RegisterDriverResponse)

```

```
err := c.cc.Invoke(ctx, DriverService_UnregisterDriver_FullMethodName, in, out, cO
pts...)
if err != nil {
    return nil, err
}
return out, nil
}

// DriverServiceServer is the server API for DriverService service.
// All implementations must embed UnimplementedDriverServiceServer
// for forward compatibility.
type DriverServiceServer interface {
    RegisterDriver(context.Context, *RegisterDriverRequest) (*RegisterDriverResponse,
error)
    UnregisterDriver(context.Context, *RegisterDriverRequest) (*RegisterDriverResponse
, error)
    mustEmbedUnimplementedDriverServiceServer()
}

// UnimplementedDriverServiceServer must be embedded to have
// forward compatible implementations.
//
// NOTE: this should be embedded by value instead of pointer to avoid a nil
// pointer dereference when methods are called.
type UnimplementedDriverServiceServer struct{}

func (UnimplementedDriverServiceServer) RegisterDriver(context.Context, *RegisterDrive
rRequest) (*RegisterDriverResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method RegisterDriver not implemen
ted")
}
func (UnimplementedDriverServiceServer) UnregisterDriver(context.Context, *RegisterDri
verRequest) (*RegisterDriverResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method UnregisterDriver not implem
ented")
}
func (UnimplementedDriverServiceServer) mustEmbedUnimplementedDriverServiceServer() {}
func (UnimplementedDriverServiceServer) testEmbeddedByValue() {}

// UnsafeDriverServiceServer may be embedded to opt out of forward compatibility for t
his service.
// Use of this interface is not recommended, as added methods to DriverServiceServer w
ill
// result in compilation errors.
type UnsafeDriverServiceServer interface {
    mustEmbedUnimplementedDriverServiceServer()
}

func RegisterDriverServiceServer(s grpc.ServiceRegistrar, srv DriverServiceServer) {
    // If the following call panics, it indicates UnimplementedDriverServiceServer was
    // embedded by pointer and is nil. This will cause panics if an
    // unimplemented method is ever invoked, so we test this at initialization
    // time to prevent it from happening at runtime later due to I/O.
    if t, ok := srv.(interface{ testEmbeddedByValue() }); ok {
        t.testEmbeddedByValue()
    }
    s.RegisterService(&DriverService_ServiceDesc, srv)
}

func _DriverService_RegisterDriver_Handler(srv interface{}, ctx context.Context, dec f
unc(interface{}) error, interceptor grpc.UnaryServerInterceptor) (interface{}, error)
{
    in := new(RegisterDriverRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(DriverServiceServer).RegisterDriver(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:    srv,
        FullMethod: DriverService_RegisterDriver_FullMethodName,
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(DriverServiceServer).RegisterDriver(ctx, req.(*RegisterDriverReque
st))
    }
    return interceptor(ctx, in, info, handler)
}

func _DriverService_UnregisterDriver_Handler(srv interface{}, ctx context.Context, dec
func(interface{}) error, interceptor grpc.UnaryServerInterceptor) (interface{}, error)
{
    in := new(RegisterDriverRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(DriverServiceServer).UnregisterDriver(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:    srv,
        FullMethod: DriverService_UnregisterDriver_FullMethodName,
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(DriverServiceServer).UnregisterDriver(ctx, req.(*RegisterDriverReq
uest))
    }
    return interceptor(ctx, in, info, handler)
}

// DriverService_ServiceDesc is the grpc.ServiceDesc for DriverService service.
// It's only intended for direct use with grpc.RegisterService,
// and not to be introspected or modified (even as a copy)
var DriverService_ServiceDesc = grpc.ServiceDesc{
    ServiceName: "driver.DriverService",
    HandlerType: (*DriverServiceServer)(nil),
    Methods: []grpc.MethodDesc{
        {
            MethodName: "RegisterDriver",
            Handler:    _DriverService_RegisterDriver_Handler,
        },
        {
            MethodName: "UnregisterDriver",
            Handler:    _DriverService_UnregisterDriver_Handler,
        },
    },
    Streams: []grpc.StreamDesc{
        {
            Metadata: "driver.proto",
        },
    },
}
```

FOLDER: shared/proto/trip/**FILE: trip.pb.go**

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//   protoc-gen-go v1.36.6
//   protoc        v5.29.3
// source: trip.proto
```

```
package trip

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
    sync "sync"
    unsafe "unsafe"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type PreviewTripRequest struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    UserID         string                      `protobuf:"bytes,1,opt,name=userID,proto3" js
on:"userID,omitempty"`
    StartLocation *Coordinate                `protobuf:"bytes,2,opt,name=startLocation,proto3" json:"startLocation,omitempty"`
    EndLocation   *Coordinate                `protobuf:"bytes,3,opt,name=endLocation,proto3" json:"endLocation,omitempty"`
    unknownFields protoimpl.UnknownFields
    sizeCache     protoimpl.SizeCache
}

func (x *PreviewTripRequest) Reset() {
    *x = PreviewTripRequest{}
    mi := &file_trip_proto_msgTypes[0]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *PreviewTripRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*PreviewTripRequest) ProtoMessage() {}

func (x *PreviewTripRequest) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[0]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use PreviewTripRequest.ProtoReflect.Descriptor instead.
func (*PreviewTripRequest) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{0}
}

func (x *PreviewTripRequest) GetUserID() string {
    if x != nil {
        return x.UserID
    }
    return ""
}

func (x *PreviewTripRequest) GetStartLocation() *Coordinate {
    if x != nil {
        return x.StartLocation
    }
    return nil
}

func (x *PreviewTripRequest) GetEndLocation() *Coordinate {
    if x != nil {
        return x.EndLocation
    }
    return nil
}

type PreviewTripResponse struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    TripID         string                  `protobuf:"bytes,1,opt,name=tripID,proto3" js
on:"tripID,omitempty"`
    Route          *Route                 `protobuf:"bytes,2,opt,name=route,proto3" js
on:"route,omitempty"`
    RideFares      []RideFare             `protobuf:"bytes,3,rep,name=rideFares,proto3"
json:"rideFares,omitempty"`
    unknownFields protoimpl.UnknownFields
    sizeCache     protoimpl.SizeCache
}

func (x *PreviewTripResponse) Reset() {
    *x = PreviewTripResponse{}
    mi := &file_trip_proto_msgTypes[1]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *PreviewTripResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*PreviewTripResponse) ProtoMessage() {}

func (x *PreviewTripResponse) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[1]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use PreviewTripResponse.ProtoReflect.Descriptor instead.
func (*PreviewTripResponse) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{1}
}

func (x *PreviewTripResponse) GetTripID() string {
    if x != nil {
        return x.TripID
    }
    return ""
}
```

```

func (x *PreviewTripResponse) GetRoute() *Route {
    if x != nil {
        return x.Route
    }
    return nil
}

func (x *PreviewTripResponse) GetRideFares() []*RideFare {
    if x != nil {
        return x.RideFares
    }
    return nil
}

type Coordinate struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    Latitude   float64                 `protobuf:"fixed64,1,opt,name=latitude,proto3`
    json:"latitude,omitempty"
    Longitude  float64                 `protobuf:"fixed64,2,opt,name=longitude,proto3`
    json:"longitude,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

func (x *Coordinate) Reset() {
    *x = Coordinate{}
    mi := &file_trip_proto_msgTypes[2]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Coordinate) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Coordinate) ProtoMessage() {}

func (x *Coordinate) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[2]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Coordinate.ProtoReflect.Descriptor instead.
func (*Coordinate) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{2}
}

func (x *Coordinate) GetLatitude() float64 {
    if x != nil {
        return x.Latitude
    }
    return 0
}

func (x *Coordinate) GetLongitude() float64 {
    if x != nil {
        return x.Longitude
    }
    return 0
}

type Geometry struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    Coordinates []Coordinate           `protobuf:"bytes,1,rep,name=coordinates,proto3`
    json:"coordinates,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

func (x *Geometry) Reset() {
    *x = Geometry{}
    mi := &file_trip_proto_msgTypes[3]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Geometry) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Geometry) ProtoMessage() {}

func (x *Geometry) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[3]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Geometry.ProtoReflect.Descriptor instead.
func (*Geometry) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{3}
}

func (x *Geometry) GetCoordinates() []Coordinate {
    if x != nil {
        return x.Coordinates
    }
    return nil
}

type Route struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    Geometry   []Geometry             `protobuf:"bytes,1,rep,name=geometry,proto3`
    json:"geometry,omitempty"
    Distance   float64                 `protobuf:"fixed64,2,opt,name=distance,proto3`
    json:"distance,omitempty"
    Duration   float64                 `protobuf:"fixed64,3,opt,name=duration,proto3`
    json:"duration,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

func (x *Route) Reset() {
    *x = Route{}
    mi := &file_trip_proto_msgTypes[4]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Route) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Route) ProtoMessage() {}

func (x *Route) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[4]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Route.ProtoReflect.Descriptor instead.
func (*Route) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{4}
}

func (x *Route) GetGeometry() []Geometry {
    if x != nil {
        return x.Geometry
    }
    return nil
}

func (x *Route) GetDistance() float64 {
    if x != nil {
        return x.Distance
    }
    return 0
}

func (x *Route) GetDuration() float64 {
    if x != nil {
        return x.Duration
    }
    return 0
}

type RideFare struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    Id         string                 `protobuf:"bytes,1,opt,name=id,proto3" js`
    on:"id,omitempty"
    UserID     string                 `protobuf:"bytes,2,opt,name=userID,proto3`
    json:"userID,omitempty"
    PackageSlug string                 `protobuf:"bytes,3,opt,name=packageSlug,proto3`
    json:"packageSlug,omitempty"
    TotalPriceInCents float64 `protobuf:"fixed64,4,opt,name=totalPriceInCents,proto3" js`
    json:"totalPriceInCents,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

func (x *RideFare) Reset() {
    *x = RideFare{}
    mi := &file_trip_proto_msgTypes[5]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *RideFare) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*RideFare) ProtoMessage() {}

func (x *RideFare) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[5]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use RideFare.ProtoReflect.Descriptor instead.
func (*RideFare) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{5}
}

func (x *RideFare) GetId() string {
    if x != nil {
        return x.Id
    }
    return ""
}

func (x *RideFare) GetUserID() string {
    if x != nil {
        return x.UserID
    }
    return ""
}

func (x *RideFare) GetPackageSlug() string {
    if x != nil {
        return x.PackageSlug
    }
    return ""
}

func (x *RideFare) GetTotalPriceInCents() float64 {
    if x != nil {
        return x.TotalPriceInCents
    }
    return 0
}

type CreateTripRequest struct {
    state      protoimpl.MessageState `protogen:"open.v1"`
    RideFareID string                 `protobuf:"bytes,1,opt,name=rideFareID,proto3`
    json:"rideFareID,omitempty"
    UserID     string                 `protobuf:"bytes,2,opt,name=userID,proto3" js`
    on:"userID,omitempty"
    unknownFields protoimpl.UnknownFields
    sizeCache    protoimpl.SizeCache
}

```



```

func (x *CreateTripRequest) Reset() {
    *x = CreateTripRequest{}
    mi := &file_trip_proto_msgTypes[6]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *CreateTripRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*CreateTripRequest) ProtoMessage() {}

func (x *CreateTripRequest) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[6]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use CreateTripRequest.ProtoReflect.Descriptor instead.
func (*CreateTripRequest) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{6}
}

func (x *CreateTripRequest) GetRideFareID() string {
    if x != nil {
        return x.RideFareID
    }
    return ""
}

func (x *CreateTripRequest) GetUserID() string {
    if x != nil {
        return x.UserID
    }
    return ""
}

type CreateTripResponse struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    TripID         string                  `protobuf:"bytes,1,opt,name=tripID,proto3" json:"tripID,omitempty"`
    Trip           *Trip                  `protobuf:"bytes,2,opt,name=trip,proto3" json:"trip,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *CreateTripResponse) Reset() {
    *x = CreateTripResponse{}
    mi := &file_trip_proto_msgTypes[7]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *CreateTripResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*CreateTripResponse) ProtoMessage() {}

func (x *CreateTripResponse) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[7]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use CreateTripResponse.ProtoReflect.Descriptor instead.
func (*CreateTripResponse) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{7}
}

func (x *CreateTripResponse) GetTripID() string {
    if x != nil {
        return x.TripID
    }
    return ""
}

func (x *CreateTripResponse) GetTrip() *Trip {
    if x != nil {
        return x.Trip
    }
    return nil
}

type Trip struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    Id             string                  `protobuf:"bytes,1,opt,name=id,proto3" json:"id,omitempty"`
    SelectedFare   *RideFare               `protobuf:"bytes,2,opt,name=selectedFare,proto3" json:"selectedFare,omitempty"`
    Route          *Route                  `protobuf:"bytes,3,opt,name=route,proto3" json:"route,omitempty"`
    Status         string                  `protobuf:"bytes,4,opt,name=status,proto3" json:"status,omitempty"`
    UserID         string                  `protobuf:"bytes,5,opt,name=userID,proto3" json:"userID,omitempty"`
    Driver         *TripDriver             `protobuf:"bytes,6,opt,name=driver,proto3" json:"driver,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *Trip) Reset() {
    *x = Trip{}
    mi := &file_trip_proto_msgTypes[8]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *Trip) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*Trip) ProtoMessage() {}

func (x *Trip) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[8]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use Trip.ProtoReflect.Descriptor instead.
func (*Trip) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{8}
}

func (x *Trip) GetId() string {
    if x != nil {
        return x.Id
    }
    return ""
}

func (x *Trip) GetSelectedFare() *RideFare {
    if x != nil {
        return x.SelectedFare
    }
    return nil
}

func (x *Trip) GetRoute() *Route {
    if x != nil {
        return x.Route
    }
    return nil
}

func (x *Trip) GetStatus() string {
    if x != nil {
        return x.Status
    }
    return ""
}

func (x *Trip) GetUserID() string {
    if x != nil {
        return x.UserID
    }
    return ""
}

func (x *Trip) GetDriver() *TripDriver {
    if x != nil {
        return x.Driver
    }
    return nil
}

// Static driver object that is used to store the driver information
type TripDriver struct {
    state          protoimpl.MessageState `protogen:"open.v1"`
    Id             string                  `protobuf:"bytes,1,opt,name=id,proto3" json:"id,omitempty"`
    Name           string                  `protobuf:"bytes,2,opt,name=name,proto3" json:"name,omitempty"`
    ProfilePicture string                  `protobuf:"bytes,3,opt,name=profilePicture,proto3" json:"profilePicture,omitempty"`
    CarPlate       string                  `protobuf:"bytes,4,opt,name=carPlate,proto3" json:"carPlate,omitempty"`
    unknownFields  protoimpl.UnknownFields
    sizeCache      protoimpl.SizeCache
}

func (x *TripDriver) Reset() {
    *x = TripDriver{}
    mi := &file_trip_proto_msgTypes[9]
    ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
    ms.StoreMessageInfo(mi)
}

func (x *TripDriver) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*TripDriver) ProtoMessage() {}

func (x *TripDriver) ProtoReflect() protoreflect.Message {
    mi := &file_trip_proto_msgTypes[9]
    if x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use TripDriver.ProtoReflect.Descriptor instead.
func (*TripDriver) Descriptor() ([]byte, []int) {
    return file_trip_proto_rawDescGZIP(), []int{9}
}

func (x *TripDriver) GetId() string {
    if x != nil {
        return x.Id
    }
    return ""
}

func (x *TripDriver) GetName() string {
    if x != nil {
        return x.Name
    }
    return ""
}

func (x *TripDriver) GetProfilePicture() string {
    if x != nil {
        return x.ProfilePicture
    }
    return ""
}

func (x *TripDriver) GetCarPlate() string {
    if x != nil {
        return x.CarPlate
    }
    return ""
}

```

```

    }.Build()
    file_trip_proto = out.File
    file_trip_proto_goTypes = nil
    file_trip_proto_depIdxs = nil
}

FILE: trip_grpc.pb.go
// Code generated by protoc-gen-go-grpc. DO NOT EDIT.
// versions:
// - protoc-gen-go-grpc v1.5.1
// - protoc v5.29.3
// source: trip.proto

package trip

import (
    context "context"
    grpc "google.golang.org/grpc"
    codes "google.golang.org/grpc/codes"
    status "google.golang.org/grpc/status"
)

// This is a compile-time assertion to ensure that this generated file
// is compatible with the grpc package it is being compiled against.
// Requires gRPC-Go v1.64.0 or later.
const _ = grpc.SupportPackageIsVersion9

const (
    TripService_PreviewTrip_FullMethodName = "/trip.TripService/PreviewTrip"
    TripService_CreateTrip_FullMethodName = "/trip.TripService/CreateTrip"
)

// TripServiceClient is the client API for TripService service.
//
// For semantics around ctx use and closing/ending streaming RPCs, please refer to https://pkg.go.dev/google.golang.org/grpc/?tab=doc#ClientConn.NewStream.
type TripServiceClient interface {
    PreviewTrip(ctx context.Context, in *PreviewTripRequest, opts ...grpc.CallOption) (*PreviewTripResponse, error)
    CreateTrip(ctx context.Context, in *CreateTripRequest, opts ...grpc.CallOption) (*CreateTripResponse, error)
}

type tripServiceClient struct {
    cc grpc.ClientConnInterface
}

func NewTripServiceClient(cc grpc.ClientConnInterface) TripServiceClient {
    return &tripServiceClient{cc}
}

func (c *tripServiceClient) PreviewTrip(ctx context.Context, in *PreviewTripRequest, opts ...grpc.CallOption) (*PreviewTripResponse, error) {
    cOpts := append([]grpc.CallOption{grpc.StaticMethod()}, opts...)
    out := new(PreviewTripResponse)
    err := c.cc.Invoke(ctx, TripService_PreviewTrip_FullMethodName, in, out, cOpts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

func (c *tripServiceClient) CreateTrip(ctx context.Context, in *CreateTripRequest, opts ...grpc.CallOption) (*CreateTripResponse, error) {
    cOpts := append([]grpc.CallOption{grpc.StaticMethod()}, opts...)
    out := new(CreateTripResponse)
    err := c.cc.Invoke(ctx, TripService_CreateTrip_FullMethodName, in, out, cOpts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

// TripServiceServer is the server API for TripService service.
// All implementations must embed UnimplementedTripServiceServer
// for forward compatibility.
type TripServiceServer interface {
    PreviewTrip(context.Context, *PreviewTripRequest) (*PreviewTripResponse, error)
    CreateTrip(context.Context, *CreateTripRequest) (*CreateTripResponse, error)
    mustEmbedUnimplementedTripServiceServer()
}

// UnimplementedTripServiceServer must be embedded to have
// forward compatible implementations.
//
// NOTE: this should be embedded by value instead of pointer to avoid a nil
// pointer dereference when methods are called.
type UnimplementedTripServiceServer struct{}

func (UnimplementedTripServiceServer) PreviewTrip(context.Context, *PreviewTripRequest) (*PreviewTripResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method PreviewTrip not implemented")
}
func (UnimplementedTripServiceServer) CreateTrip(context.Context, *CreateTripRequest) (*CreateTripResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method CreateTrip not implemented")
}
func (UnimplementedTripServiceServer) mustEmbedUnimplementedTripServiceServer() {}
func (UnimplementedTripServiceServer) testEmbeddedByValue() {}

// UnsafeTripServiceServer may be embedded to opt out of forward compatibility for this service.
// Use of this interface is not recommended, as added methods to TripServiceServer will
// result in compilation errors.
type UnsafeTripServiceServer interface {
    mustEmbedUnimplementedTripServiceServer()
}

func RegisterTripServiceServer(s grpc.ServiceRegistrar, srv TripServiceServer) {
    // If the following call panics, it indicates UnimplementedTripServiceServer was
    // embedded by pointer and is nil. This will cause panics if an
    // unimplemented method is ever invoked, so we test this at initialization
    // time to prevent it from happening at runtime later due to I/O.
    if t, ok := srv.(interface{ testEmbeddedByValue() }); ok {
        t.testEmbeddedByValue()
    }
    s.RegisterService(&TripService_ServiceDesc, srv)
}

func _TripService_PreviewTrip_Handler(srv interface{}, ctx context.Context, dec func(interface{}) error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(PreviewTripRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {

```

```

        return srv.(TripServiceServer).PreviewTrip(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:    srv,
        FullMethod: TripService_PreviewTrip_FullMethodName,
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(TripServiceServer).PreviewTrip(ctx, req.(*PreviewTripRequest))
    }
    return interceptor(ctx, in, info, handler)
}

func _TripService_CreateTrip_Handler(srv interface{}, ctx context.Context, dec func(interface{}) error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(CreateTripRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(TripServiceServer).CreateTrip(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:    srv,
        FullMethod: TripService_CreateTrip_FullMethodName,
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(TripServiceServer).CreateTrip(ctx, req.(*CreateTripRequest))
    }
    return interceptor(ctx, in, info, handler)
}

// TripService_ServiceDesc is the grpc.ServiceDesc for TripService service.
// It's only intended for direct use with grpc.RegisterService,
// and not to be introspected or modified (even as a copy)
var TripService_ServiceDesc = grpc.ServiceDesc{
    ServiceName: "trip.TripService",
    HandlerType: (*TripServiceServer)(nil),
    Methods: []grpc.MethodDesc{
        {
            MethodName: "PreviewTrip",
            Handler:    _TripService_PreviewTrip_Handler,
        },
        {
            MethodName: "CreateTrip",
            Handler:    _TripService_CreateTrip_Handler,
        },
    },
    Streams: []grpc.StreamDesc{
        {
            Metadata: "trip.proto",
        },
    },
}

```

FOLDER: shared/retry/**FILE: retry.go**

```

/*
Package retry provides a simple retry mechanism with exponential backoff.
It is as abstract as possible to allow for different retry strategies.
*/
package retry

import (
    "context"
    "log"
    "time"
)

type Config struct {
    MaxRetries int
    InitialWait time.Duration
    MaxWait    time.Duration
}

// DefaultConfig returns a Config with sensible default values
func DefaultConfig() Config {
    return Config{
        MaxRetries: 3,
        InitialWait: 1 * time.Second,
        MaxWait:    10 * time.Second,
    }
}

// WithBackoff executes the given operation with exponential backoff retry logic
func WithBackoff(ctx context.Context, cfg Config, operation func() error) error {
    var err error
    wait := cfg.InitialWait

    for attempt := 0; attempt <= cfg.MaxRetries; attempt++ {
        if attempt > 0 {
            log.Printf("Retry attempt %d/%d after %v", attempt, cfg.MaxRetries, wait)

            select {
            case <-ctx.Done():
                return ctx.Err()
            case <-time.After(wait):
            }

            // Exponential backoff with max wait cap
            wait *= 2
            if wait > cfg.MaxWait {
                wait = cfg.MaxWait
            }
        }

        if err = operation(); err == nil {
            return nil
        }

        log.Printf("Operation failed (attempt %d/%d): %v", attempt+1, cfg.MaxRetries, err)
    }

    return err
}

```

FOLDER: shared/tracing/**FILE: grpc.go**

```

package tracing

import (
    "go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc"
    "go.opentelemetry.io/otel"
    "google.golang.org/grpc"
    "google.golang.org/grpc/stats"
)

func WithTracingInterceptors() []grpc.ServerOption {
    return []grpc.ServerOption{
        grpc.StatsHandler(newServerHandler()),
    }
}

```

```

}

func DialOptionsWithTracing() []grpc.DialOption {
    return []grpc.DialOption{
        grpc.WithStatsHandler(newClientHandler()),
    }
}

func newClientHandler() stats.Handler {
    return otelgrpc.NewClientHandler(otelgrpc.WithTracerProvider(otel.GetTracerProvider()))
}

func newServerHandler() stats.Handler {
    return otelgrpc.NewServerHandler(otelgrpc.WithTracerProvider(otel.GetTracerProvider()))
}

```

FILE: http.go

package tracing

```

import (
    "net/http"

    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
)

func WrapHandlerFunc(handler http.HandlerFunc, operation string) http.Handler {
    return otelhttp.NewHandler(handler, operation)
}

```

FILE: rabbitmq.go

package tracing

```

import (
    "context"
    "encoding/json"

    "ride-sharing/shared/contracts"

    amqp "github.com/rabbitmq/amqp091-go"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/codes"
    "go.opentelemetry.io/otel/trace"
)

// amqpHeadersCarrier implements the TextMapCarrier interface for AMQP headers
type amqpHeadersCarrier amqp.Table

func (c amqpHeadersCarrier) Get(key string) string {
    if v, ok := c[key]; ok {
        if s, ok := v.(string); ok {
            return s
        }
    }
    return ""
}

func (c amqpHeadersCarrier) Set(key string, value string) {
    c[key] = value
}

func (c amqpHeadersCarrier) Keys() []string {
    keys := make([]string, 0, len(c))
    for k := range c {
        keys = append(keys, k)
    }
    return keys
}

// TracedPublisher wraps the RabbitMQ publish function with tracing
func TracedPublisher(ctx context.Context, exchange, routingKey string, msg amqp.Publishing, publish func(context.Context, string, string, amqp.Publishing) error) error {
    tracer := otel.GetTracerProvider().Tracer("rabbitmq")

    ctx, span := tracer.Start(ctx, "rabbitmq.publish",
        trace.WithAttributes(
            attribute.String("messaging.destination", exchange),
            attribute.String("messaging.routing_key", routingKey),
        ),
    )
    defer span.End()

    // Try to extract and add message details to span (map[string]any if you don't know the type)
    var msgBody contracts.AmqpMessage
    if err := json.Unmarshal(msg.Body, &msgBody); err == nil {
        if msgBody.OwnerID != "" {
            span.SetAttributes(attribute.String("messaging.owner_id", msgBody.OwnerID))
        }
    }

    // Inject trace context into message headers
    if msg.Headers == nil {
        msg.Headers = make(amqp.Table)
    }
    carrier := amqpHeadersCarrier(msg.Headers)
    otel.GetTextMapPropagator().Inject(ctx, carrier)
    msg.Headers = amqp.Table(carrier)

    if err := publish(ctx, exchange, routingKey, msg); err != nil {
        span.SetStatus(codes.Error, err.Error())
        return err
    }

    return nil
}

// TracedConsumer wraps the RabbitMQ message handler with tracing
func TracedConsumer(delivery amqp.Delivery, handler func(context.Context, amqp.Delivery) error) error {
    // Extract trace context from message headers
    carrier := amqpHeadersCarrier(delivery.Headers)
    ctx := otel.GetTextMapPropagator().Extract(context.Background(), carrier)

    tracer := otel.GetTracerProvider().Tracer("rabbitmq")

    ctx, span := tracer.Start(ctx, "rabbitmq.consume",
        trace.WithAttributes(
            attribute.String("messaging.destination", delivery.Exchange),
            attribute.String("messaging.routing_key", delivery.RoutingKey),
        ),
    )
    defer span.End()

    // Try to extract and add message details to span (map[string]any if you don't know the type)
    var msgBody contracts.AmqpMessage

```

```
    if err := json.Unmarshal(delivery.Body, &msgBody); err == nil {
        if msgBody.OwnerID != "" {
            span.SetAttributes(attribute.String("messaging.owner_id", msgBody.OwnerID))
        }
    }

    if err := handler(ctx, delivery); err != nil {
        span.SetStatus(codes.Error, err.Error())
        return err
    }

    return nil
}
```

FILE: tracing.go

```
package tracing

import (
    "context"
    "fmt"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/exporters/jaeger"
    "go.opentelemetry.io/otel/propagation"
    "go.opentelemetry.io/otel/sdk/resource"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    semconv "go.opentelemetry.io/otel/semconv/v1.26.0"
    "go.opentelemetry.io/otel/trace"
)

type Config struct {
    ServiceName string
    Environment string
    JaegerEndpoint string
}

func InitTracer(cfg Config) (func(context.Context) error, error) {
    // Exporter
    traceExporter, err := newExporter(cfg.JaegerEndpoint)
    if err != nil {
        return nil, err
    }

    // Trace Provider
    traceProvider, err := newTraceProvider(cfg, traceExporter)
    if err != nil {
        return nil, err
    }
    otel.SetTracerProvider(traceProvider)

    // Propagator
    prop := newPropagator()
    otel.SetTextMapPropagator(prop)

    return traceProvider.Shutdown, nil
}

func GetTracer(name string) trace.Tracer {
    return otel.GetTracerProvider().Tracer(name)
}

func newExporter(endpoint string) (sdktrace.SpanExporter, error) {
    return jaeger.New(jaeger.WithCollectorEndpoint(jaeger.WithEndpoint(endpoint)))
}

func newPropagator() propagation.TextMapPropagator {
    return propagation.NewCompositeTextMapPropagator(
        propagation.TraceContext{},
        propagation.Baggage{},
    )
}

func newTraceProvider(cfg Config, exporter sdktrace.SpanExporter) (*sdktrace.TracerProvider, error) {
    res, err := resource.New(context.Background(),
        resource.WithAttributes(
            semconv.ServiceNameKey.String(cfg.ServiceName),
            semconv.DeploymentEnvironmentKey.String(cfg.Environment),
        ),
    )
    if err != nil {
        return nil, fmt.Errorf("failed to create resource: %w", err)
    }

    traceProvider := sdktrace.NewTracerProvider(
        sdktrace.WithBatcher(exporter),
        sdktrace.WithResource(res),
    )

    return traceProvider, nil
}
```

FOLDER: shared/types/**FILE: types.go**

```
package types

type Route struct {
    Distance float64 `json:"distance"`
    Duration float64 `json:"duration"`
    Geometry []Geometry `json:"geometry"`
}

type Geometry struct {
    Coordinates [][]Coordinate `json:"coordinates"`
}

type Coordinate struct {
    Latitude float64 `json:"latitude"`
    Longitude float64 `json:"longitude"`
}
```

FOLDER: shared/util/**FILE: util.go**

```
package util

import "fmt"

// GetRandomAvatar returns a random avatar URL from the randomuser.me API
func GetRandomAvatar(index int) string {
    return fmt.Sprintf("https://randomuser.me/api/portraits/lego/%d.jpg", index)
}
```

FOLDER: tools/**FILE: create_service.go**

```
package main

import (
```

```
    "flag"
    "fmt"
    "os"
    "path/filepath"
)

func main() {
    serviceName := flag.String("name", "", "Name of the service (e.g., user, payment)")

    flag.Parse()

    if *serviceName == "" {
        fmt.Println("Please provide a service name using -name flag")
        os.Exit(1)
    }

    // Create service directory structure
    basePath := filepath.Join("services", *serviceName+"-service")
    dirs := []string{
        "cmd/",
        "internal/domain/",
        "internal/service/",
        "internal/infrastructure/events/",
        "internal/infrastructure/grpc/",
        "internal/infrastructure/repository/",
        "pkg/types",
    }

    for _, dir := range dirs {
        fullPath := filepath.Join(basePath, dir)
        if err := os.MkdirAll(fullPath, 0755); err != nil {
            fmt.Printf("Error creating directory %s: %v\n", dir, err)
            os.Exit(1)
        }
    }

    // Create an empty README.md
    readmePath := filepath.Join(basePath, "README.md")
    readmeContent := fmt.Sprintf(`# %s service
```

This service handles all %s-related operations in the system.

Architecture

The service follows Clean Architecture principles with the following structure:

```
` + "``" + `
services/%s-service/
├── cmd/                # Application entry points
│   └── main.go         # Main application setup
├── internal/           # Private application code
│   ├── domain/         # Business domain models and interfaces
│   ├── service/        # Business logic implementation
│   │   └── %s.go        # Service implementations
│   ├── infrastructure/ # External dependencies implementations (abstractions)
│   │   ├── events/     # Event handling (RabbitMQ)
│   │   ├── grpc/       # gRPC server handlers
│   │   └── repository/ # Data persistence
│   └── pkg/            # Public packages
│       ├── types/      # Shared types and models
│       └── README.md   # This file
` + "``" + `
```

Layer Responsibilities

- Domain Layer** (` + "`internal/domain/" + `)
 - Contains business domain interfaces
 - Defines contracts for repositories and services
 - Pure business logic, no implementation details
- Service Layer** (` + "`internal/service/" + `)
 - Implements business logic
 - Uses repository interfaces
 - Coordinates between different parts of the system
- Infrastructure Layer** (` + "`internal/infrastructure/" + `)
 - ` + "`repository/" + `: Implements data persistence
 - ` + "`events/" + `: Handles event publishing and consuming
 - ` + "`grpc/" + `: Handles gRPC communication
- Public Types** (` + "`pkg/types/" + `)
 - Contains shared types and models
 - Can be imported by other services

Key Benefits

- Dependency Inversion**: Services depend on interfaces, not implementations
- Separation of Concerns**: Each layer has a specific responsibility
- Testability**: Easy to mock dependencies for testing
- Maintainability**: Clear boundaries between components
- Flexibility**: Easy to swap implementations without affecting business logic

```
` , *serviceName, *serviceName, *serviceName)

if err := os.WriteFile(readmePath, []byte(readmeContent), 0644); err != nil {
    fmt.Printf("Error creating README.md: %v\n", err)
    os.Exit(1)
}
```

```
    fmt.Printf("Successfully created %s service structure in %s\n", *serviceName, basePath)
    fmt.Println("\nDirectory structure created:")
    fmt.Printf(`
services/%s-service/
├── cmd/                # Application entry points
│   └── main.go         # Main application setup
├── internal/           # Private application code
│   ├── domain/         # Business domain models and interfaces
│   ├── %s.go           # Core domain interfaces
│   ├── service/        # Business logic implementation
│   │   └── %s.go        # Service implementations
│   ├── infrastructure/ # External dependencies implementations (abstractions)
│   │   ├── events/     # Event handling (RabbitMQ)
│   │   ├── grpc/       # gRPC server handlers
│   │   └── repository/ # Data persistence
│   └── pkg/            # Public packages
│       ├── types/      # Shared types and models
│       └── README.md   # This file
` , *serviceName, *serviceName)
}
```